

세션 2 · MongoDB 운영

Replica Set · 선출 · Index · Backup/Restore · 권한(Role) · 장애 전환 · Sharding

대상 버전

MongoDB 8.3.x

OS

Ubuntu 24.04 LTS

실습 환경

Azure

01

인프라 구축

스택 구성

Terraform 실행

자원 확인

네트워크 규칙

노드 접속

02-mongodb 스택

노드	사양	역할
mongo-1 mongo-2 mongo-3	Standard_E2bds_v5 2 vCPU / 16 GiB 데이터 디스크 128 GiB	MongoDB 노드
ops-1	Standard_D2ds_v5 2 vCPU / 8 GiB 데이터 디스크 64 GiB	명령 실행, Prometheus·Grafana

세션 1 스택과 다른 것은 서비스 노드의 데이터 디스크 하나입니다.
 MongoDB 는 데이터를 디스크에 두고, 세션 1 의 Redis 는 메모리에 둡니다.

인스턴스 스펙 근거

항목	내용
Standard_E2bds_v5 2 vCPU / 16 GiB	working set 과 인덱스를 캐시에 담을 수 있는 vCPU당 메모리를 기준으로 고른 가장 작은 크기
OS 와 exporter	약 1 GiB
WiredTiger 캐시	약 7.31 GiB. 메모리의 절반가량
캐시 밖 사용	약 2 GiB. 연결마다 쓰는 버퍼, 정렬과 집계
파일시스템 캐시 여유	약 5.3 GiB. 압축된 채로 남아 디스크 재읽기를 줄임

남는 메모리도 비어 있는 것이 아닙니다.

WiredTiger 가 쓰지 않는 메모리를 OS 가 파일시스템 캐시로 씁니다.

변수 파일 작성

```
subscription_id      = "000000000-0000-0000-0000-000000000000"
resource_group_name = "rkm-<계정 이름>-mongo-rg"
location             = "koreacentral"
allowed_cidrs        = ["xxx.xxx.xxx.xxx/32"]
ssh_public_key_path  = "~/.ssh/rkm.pub"
```

- **subscription_id**: `az account show --query id -o tsv`
- **resource_group_name**: `rkm-<계정 이름>-mongo-rg`. 형식이 다르면 `apply` 전에 멈춤
- **allowed_cidrs**: `curl -s ifconfig.me` 로 확인한 자기 PC 공인 IP

구독 ID와 키는 저장소에 커밋하지 않습니다.

`session.tfvars` 는 `.gitignore` 에 들어 있습니다. 예제 파일만 저장소에 둡니다.

스택 생성

- 1 `cd lab/terraform/02-mongodb`
- 2 `cp session.tfvars.example session.tfvars` 후 값 채우기
- 3 `terraform init`
- 4 `terraform plan -var-file=session.tfvars` 로 자원 수 확인
- 5 `terraform apply -var-file=session.tfvars`
- 6 `terraform output` 으로 ops 노드 공인 IP 확인

Terraform 명령만 자기 PC 에서 실행합니다.

나머지 명령은 ops 노드에서 실행합니다. 챕터 12 의 노드 증설과 마지막 정리에서만 다시 자기 PC 로 돌아옵니다.

완료 판정

`Apply complete!` 로 끝나고 출력 세 개에 값이 전부 채워지면 완료입니다.

생성되는 자원

```
# module.mongo["mongo-1"].azurerm_linux_virtual_machine.this will be created
# module.mongo["mongo-1"].azurerm_managed_disk.data[0] will be created
# module.ops.azurerm_public_ip.this[0] will be created
...
```

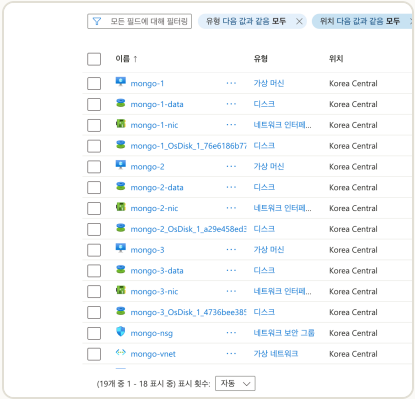
Plan: 27 to add, 0 to change, 0 to destroy.

- VM 4대: mongo-1 · mongo-2 · mongo-3 과 ops-1
- 데이터 디스크 4개: 서비스 노드 128 GiB, ops 노드 64 GiB
- 네트워크: VNet 하나, 서브넷 하나, 네트워크 보안 그룹 하나. 공인 IP 는 ops-1 에만

포털 확인 항목

확인 대상	포털 메뉴	확인 내용
자원 목록	리소스 그룹	VM 4대, 디스크, NIC, NSG, VNet
VM과 영역	가상 머신	서비스 노드 3대가 영역 1·2·3에 하나씩
디스크	디스크	128 GiB, 3,000 IOPS, 125 MB/s
인바운드 규칙	네트워크 보안 그룹	22·3000·9090이 현재 PC 공인 IP로 제한

생성 결과 확인



리소스 그룹의 자원 목록

☐	이름 ↑	리소스 그룹	위치	상태	운영 체제	크기	디스크	가용성 영역
☐	mongo-1	... rkm-verify-mongo-rg	Korea Central	실행 중	Linux	Standard_E2bds_v5	2	1
☐	mongo-2	... rkm-verify-mongo-rg	Korea Central	실행 중	Linux	Standard_E2bds_v5	2	2
☐	mongo-3	... rkm-verify-mongo-rg	Korea Central	실행 중	Linux	Standard_E2bds_v5	2	3
☐	ops-1	... rkm-verify-mongo-rg	Korea Central	실행 중	Linux	Standard_D2ds_v5	2	1

VM 목록과 가용성 영역 열

데이터 디스크 확인



크기

크기	128GiB
스토리지 유형	프리미엄 SSD v2 LRS
IOPS	3000
처리량(MB/s)	125
디스크 계층	-
논리 섹터 크기(바이트)	4096

데이터 디스크 설정

- **용량 128 GiB:** MongoDB 데이터 전용. OS 디스크와 별도로 붙는 관리 디스크
- **IOPS 3,000 · 처리량 125 MB/s:** Premium SSD v2 의 무료 최저값

ops 노드 접속

```
# Git Bash
ssh -i ~/.ssh/rkm azureuser@<ops 공인 IP>
```

- 접속 대상은 ops 노드뿐. 서비스 노드에는 공인 IP 가 없음
- 키의 출처: 세션 1에서 `ssh-keygen` 으로 만든 자기 키. `session.tfvars` 의 `ssh_public_key_path` 가 그 공개키를 가리킴

PuTTY 를 쓰면 키를 `.ppk` 로 변환해야 합니다.

Git Bash 의 `ssh` 는 변환 없이 그대로 씁니다.

서비스 노드 접근 방식

```
# ansible 명령은 이 자리에서 실행 (인벤토리가 여기 있음)
cd ~/ansible
```

```
# 방법 1. ops 노드에서 원격으로 명령만 전달 (기본)
ansible mongo -m ping
ansible mongo -a "uptime"
```

```
# 방법 2. 노드 안에서 해야 하는 작업만 직접 접속
ssh 10.0.1.6
```

이 세션의 명령은 모두 ops 노드에서 실행합니다.

세 노드를 오가지 않아도 되고, 같은 명령을 세 노드에 한 번에 보낼 수 있습니다.

02

MongoDB 구성

인벤토리

디스크 마운트

관측 스택

구성 단계가 만드는 상태

role	수행 작업	필요한 이유
common	디스크 마운트, readahead 축소, THP 비활성	데이터 경로를 OS 디스크에서 떼고, 랜덤 IO 에 맞춤
mongodb	설치와 <code>mongod.conf</code> 배포	세 노드가 같은 설정으로 기동함
mongodb	keyFile 생성과 배치	나중에 인증을 켤 때 필요
mongodb	ops 노드에 <code>mongosh</code> · <code>mongodb-database-tools</code> · <code>mongos</code> 설치. 서비스 노드에도 <code>mongosh</code> 설치	명령은 ops 노드에서 실행. 챕터 12 의 초기화만 노드 안에 서 치므로 서비스 노드에도 셸이 필요
monitoring	exporter 와 Prometheus·Grafana	이 세션에서 바로 확인

이 단계에서는 단독 `mongod` 세 대가 기동합니다.
복제 설정은 넣지 않습니다. 챕터 06 실습에서 설정을 추가하고 초기화합니다.

인벤토리 노드 목록

```
[mongo]
mongo-1 ansible_host=10.0.1.5
mongo-2 ansible_host=10.0.1.6
mongo-3 ansible_host=10.0.1.7

[ops]
ops-1 ansible_connection=local

[all:vars]
ansible_user=azureuser
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

- 채우는 주체: Terraform
- 받는 시점: ops 노드 부팅 시점
- **[ops]** : **local** 연결

mongod.conf 주요 설정

```
storage:
  dbPath: {{ mongodb_dbpath }}

systemLog:
  destination: file
  path: {{ mongodb_logpath }}
  logAppend: true
```

- 배포 위치: 세 MongoDB 노드의 `/etc/mongod.conf`. Ansible 플레이북이 배포하므로 직접 쓰지 않음
- `storage.dbPath`: 데이터 디스크 마운트 지점
- `systemLog`: 로그 파일 경로. `logAppend` 로 재기동 때 덮어쓰지 않음

구성 실행

- 1 `cd ~/ansible`
- 2 `ansible mongo -m ping` 으로 서비스 노드 3대 연결 확인
- 3 `ansible-playbook play-mongodb.yml` 실행
- 4 `ansible mongo -a "systemctl is-active mongod"` 로 기동 확인

```
mongo-1 : ok=30  changed=22  failed=0
mongo-2 : ok=30  changed=21  failed=0
mongo-3 : ok=30  changed=21  failed=0
ops-1   : ok=18  changed=14  failed=0
```

완료 판정

네 행이 전부 `failed=0` 이면 완료입니다.

대시보드에서 확인

- 1 ops 노드의 9090 포트로 Prometheus 접속. **Status → Targets** 에서 exporter 가 전부 **UP** 인지 확인. 추가 직후에는 **UNKNOWN** 으로 표시됩니다
- 2 같은 노드의 3000 포트로 Grafana 접속. 초기 계정은 **admin / admin**
- 3 **실습** 폴더의 대시보드에서 세 노드의 CPU·메모리와 MongoDB 연결 수 확인

Prometheus Alerts Graph Status ▾ Help

Targets

All Unhealthy Collapse All

mongodb (3/3 up) [show less](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://10.0.1.5:9216/metrics	UP	instance="10.0.1.5:9216" job="mongodb" services="mongodb"	13.375s ago	351.1ms	
http://10.0.1.6:9216/metrics	UP	instance="10.0.1.6:9216" job="mongodb" services="mongodb"	11.829s ago	355.6ms	
http://10.0.1.7:9216/metrics	UP	instance="10.0.1.7:9216" job="mongodb" services="mongodb"	8.831s ago	337.1ms	

node (3/3 up) [show less](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://10.0.1.5:9100/metrics	UP	instance="10.0.1.5:9100" job="node" services="mongodb"	9.525s ago	66.01ms	
http://10.0.1.6:9100/metrics	UP	instance="10.0.1.6:9100" job="node" services="mongodb"	8.209s ago	71.2ms	
http://10.0.1.7:9100/metrics	UP	instance="10.0.1.7:9100" job="node" services="mongodb"	9.53s ago	70.56ms	

Targets. **node** 와 **mongodb** 가 각각 3/3 up



Grafana 대시보드

여기서는 지표가 보이는 것까지만 확인합니다.

03

문서 모델과 저장 구조

문서 모델

BSON

스토리지 엔진

B-tree

페이지

압축

문서와 BSON

```
{
  _id: ObjectId("..."),
  customerId: "C000123",
  status: "paid",
  amount: 45000,
  createdAt: ISODate("2026-08-01"),
  items: [ "A-1", "B-2" ],
  ship: { city: "seoul" }
}
```

- **BSON**: Binary JSON. JSON 의 구조를 바이너리로 담은 형식
- **JSON 에 없는 타입**: `ObjectId`, `Date`, `Decimal128`, 정수 폭 구분
- **길이 정보의 선행 배치**: 통째로 파싱하지 않고 건너뛴
- **문서 하나의 상한**: 16 MB

`_id` 는 모든 문서에 반드시 있습니다.

넣지 않으면 서버가 `ObjectId` 를 만들어 붙이고, 이 필드에는 유일 인덱스가 자동으로 생깁니다.

Embedding과 Reference

```
// Embedding
{ _id: ObjectId("..."), customerId: "C000123",
  ship: { city: "seoul",
    zip: "06234", phone: "..." } }
```

```
// Reference
{ _id: ObjectId("..."), customerId: "C000123",
  shipId: ObjectId("...") }
```

방식	성질	고르는 기준
Embedding	한 문서 안에 중첩	함께 읽고 함께 고치며 크기를 예측할 수 있는 경우
Reference	다른 문서의 <code>_id</code> 를 저장	따로 조회하거나 한쪽이 계속 늘어나는 경우

판단 기준은 접근 패턴입니다.

함께 읽는 데이터는 한 문서에 넣고, 따로 쓰이거나 한쪽이 계속 늘어나면 `_id` 로 가리킵니다.

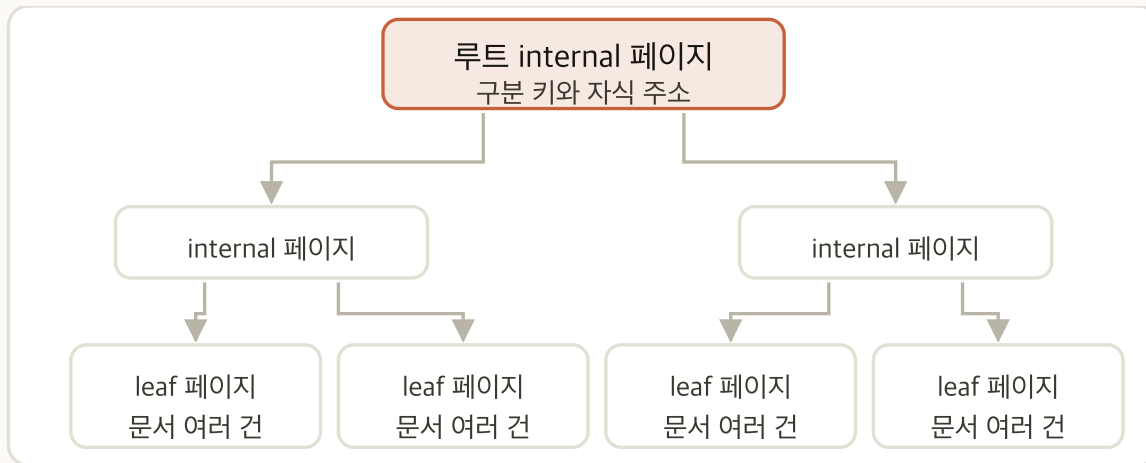
스토리지 엔진의 역할



- **질의 계층의 역할**: 질의를 해석하고 어떤 인덱스를 쓸지 정하며 권한을 확인
- **엔진의 역할**: 디스크에 어떤 형식으로 놓을지, 동시 접근을 어떻게 다룰지, 메모리에 무엇을 얼마나 둘지, 프로세스가 정지해도 무엇이 남을지를 정함
- **기본 엔진이 된 시점**: MongoDB 3.2 부터. MMAPv1 은 4.2 에서 제거됨

운영에서 마주치는 문제의 상당수가 엔진 층에서 생깁니다.
쓰기 지연, 메모리 압박, 재기동 시간은 전부 이 층의 동작으로 설명됩니다.

컬렉션 B-tree의 구조



컬렉션 파일 하나에 B-tree 전체가 저장됨

- **페이지**: WiredTiger 가 디스크와 메모리 사이를 오갈 때 쓰는 단위. **문서 하나를 읽어도 그 페이지 전체가 캐시로 올라옴**
- **컬렉션 파일**: 루트·internal·leaf 가 전부 그 안에 있고, **문서는 leaf 페이지에 들어 있음**
- **인덱스 파일**: 인덱스마다 따로 생김. 셋 만들면 파일도 셋
- **컬렉션 B-tree 키**: 내부 번호인 `RecordId`
- **인덱스 B-tree 키**: 인덱스를 건 필드의 값

인덱스를 하나 만드는 것은 트리를 하나 더 만드는 것입니다.

디스크와 캐시를 따로 쓰고 쓰기마다 함께 갱신됩니다. 쓰지 않는 인덱스를 지우는 근거가 여기 있습니다.

페이지 구조

항목	internal	leaf
저장 내용	구분 키와 자식 주소	문서 여러 건

- **구분 키**: 자식들의 키 범위를 가르는 경계값. 길잡이라서 작고, internal 하나가 자식을 수백 개까지 가리킴
- **페이지당 문서 수**: 186 바이트 문서 기준으로 32 KB 페이지에 150건 안팎
- **읽기의 지역성**: 이웃 문서가 함께 캐시로 올라옴

자주 갱신되는 작은 필드를 큰 문서에 두지 않습니다.

읽기도 쓰기도 단위가 문서가 아니라 페이지입니다. 조회수 하나를 올려도 그 페이지 전체가 다시 쓰입니다. 문서를 작게 유지하면 한 페이지에 담기는 문서 수가 늘어, 같은 캐시 용량으로 읽을 수 있는 양이 커집니다.

문서 하나의 크기

```

{
  _id:      ObjectId("64f8a123bc45de6789012345"),
  customerId: "C003456",
  status:   "paid",
  amount:   48064,
  createdAt: ISODate("2026-09-01T07:00:00Z"),
  region:   "seoul",
  items:    2,
  note:     "order 123456 placed by ..."
}
```

필드	타입	바이트
_id	ObjectId	17
문자열	7 · 4 · 5 · 50자	120
정수	amount · items	23
날짜	createdAt	19
문서 머리와 종료	길이 4 · 종료 1	5
합계		184

BSON 은 필드마다 타입 1바이트와 이름을 값 앞에 붙입니다. 이름을 줄이면 한 페이지에 담기는 문서 수가 늘어납니다.

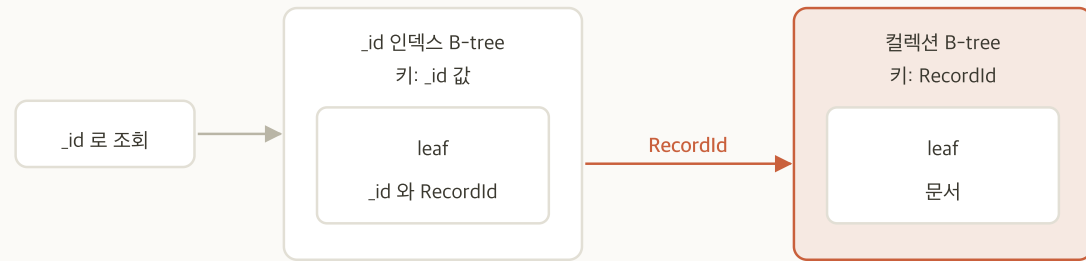
B-tree 선택 근거

기준	순차 저장	B-tree
키로 찾기	처음부터 훑음	몇 단계 내려가면 닿음
범위 조회	정렬이 없어 전체를 훑음	정렬돼 있어 leaf 를 옆으로 훑음
중간 삽입	정렬을 유지하려면 뒤를 전부 밀어야 함	그 페이지만 둘로 쪼갬

- fan-out (F): 페이지 크기 ÷ (구분 키 + 자식 주소). 컬렉션은 키가 RecordId 라 거의 일정하고, 인덱스는 건 필드의 값 길이에 따라 갈림
- 페이지당 문서 수 (L): leaf 하나에 담기는 문서 수. 150건 안팎
- 깊이: 문서 N 건이면 대략 $\log_F(N \div L)$ 단. F 160 · L 150 이면 3단에 약 380만 건, 4단에 약 6억 건

조회가 느려진 원인은 데이터가 늘어난 것이 아닙니다. 건수가 백 배 늘어도 깊이는 한 단입니다. 인덱스를 못 쓰거나 캐시에서 밀려난 것입니다.

RecordId와 _id



조회가 두 트리를 차례로 탐

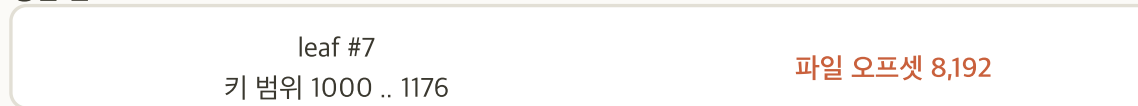
- **키 길이:** `_id` 를 저장 키로 쓰면 구분 키가 커져 fan-out 이 줄고 트리가 깊어짐. `RecordId` 는 고정 길이 정수
- **보조 인덱스의 목적지:** `RecordId` 를 가리키므로 조회가 두 번에 끝남
- **삽입 위치:** `RecordId` 는 단조 증가라 삽입이 오른쪽 끝에만 붙음
- **불변:** `RecordId` 는 문서가 있는 동안 바뀌지 않아 보조 인덱스를 고칠 일이 없음

`_id` 를 무작위 값으로 잡으면 삽입이 트리 전체에 흩어집니다.

`ObjectId` 는 시각이 앞자리라 계속 커지지만 UUID v4 는 무작위입니다. 삽입이 흩어지면 그만큼 여러 곳의 페이지가 dirty 가 됩니다. 보조 인덱스의 키도 같은 기준으로 고릅니다.

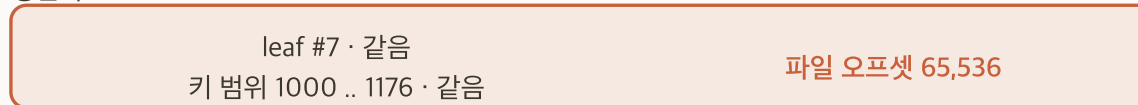
컬렉션 B-tree 에서의 갱신

갱신 전



필드 하나를 고친 뒤 eviction 또는 체크포인트

갱신 후



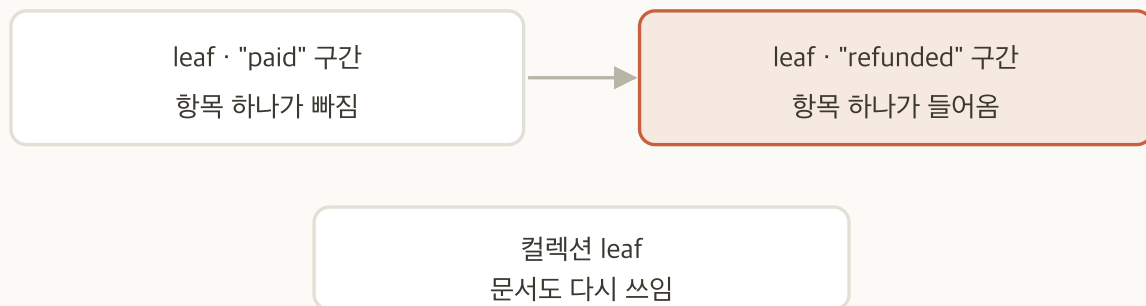
트리에서의 자리는 그대로. 파일 안의 자리만 바뀜

- **갱신 단위:** leaf 페이지 하나. 문서 한 건이 아님
- **페이지 내용:** 같은 페이지의 이웃 문서까지 **통째로 다시 직렬화** 됨
- **파일 위치:** 원래 자리를 **덮어쓰지 않고** 빈 자리에 새로 놓임. 옛 자리는 빈 구간으로 남음
- **트리 위치:** 키인 `RecordId` 가 바뀌지 않아 키 범위가 그대로. 부모는 자식의 새 파일 주소만 고쳐 씀

수정한 문서가 포함된 페이지가 통째로 다시 만들어집니다.

인덱스 B-tree 에서의 갱신

status 를 paid 에서 refunded 로 고침



- **키 변경:** 인덱스 B-tree 의 키는 그 필드의 값
- **항목의 이동:** 옛 값의 leaf 에서 빠지고 새 값의 leaf 로 들어감
- **걸리지 않은 필드:** 그 필드에 인덱스가 없으면 인덱스 페이지는 손대지 않음

자주 바뀌는 필드에는 인덱스를 신중히 합니다.

문서 하나를 고칠 때 **고친 필드에 걸린 인덱스마다** 페이지 둘이 dirty 가 됩니다. 인덱스가 넷이고 그중 둘이 걸린 필드라면 컬렉션 하나를 더해 페이지 다섯입니다.

압축 방식

컬렉션 데이터의 블록 압축	특성	고르는 기준
snappy	기본값. 빠르고 압축률은 보통	대부분의 운영 구성
zstd	압축률이 높고 CPU를 더 씀	디스크가 비싸고 CPU에 여유가 있을 때
없음	압축하지 않음	이미 압축된 데이터를 저장할 때

- **압축 단위:** 문서 하나가 아니라 leaf 페이지 전체. 압축한 뒤 디스크에 씀

블록 압축률을 높여도 캐시 여유는 늘지 않습니다.
컬렉션 데이터는 캐시에 올라올 때 압축이 풀리기 때문입니다.

저장 구조가 정하는 설계 판단

설계 항목	근거가 되는 성질	확인 기준
문서를 나눌 것인가	페이지 단위 읽기·쓰기와 쓰기 증폭	자주 갱신되는 필드가 큰 문서에 섞여 있는가
인덱스를 몇 개 둘 것인가	인덱스마다 트리와 파일. 고친 필드에 걸린 인덱스마다 페이지 둘	그 인덱스가 실제로 쓰이고 있는가
어떤 필드를 키로 잡을 것인가	단조 증가 키와 무작위 키의 삽입 위치 차이	삽입이 한곳에 붙는가 흩어지는가
배열을 문서에 담을 것인가	문서 상한 16 MB, 페이지 상한 32 KB	배열이 계속 늘어나는가

엔진의 동작을 아는 것이 목적이 아닙니다.

위 넷은 코드를 쓰기 전에 정해집니다. 나중에 바꾸려면 데이터를 옮겨야 합니다.

seed-mongo.sh 적재 스크립트

```
# ops 노드에서
MONGO_HOST=10.0.1.5 ~/loadgen/seed-mongo.sh           # lab.orders 에 20만 건
MONGO_HOST=10.0.1.5 ~/loadgen/seed-mongo.sh 50000     # 건수 지정
MONGO_HOST=10.0.1.5 ~/loadgen/seed-mongo.sh 50000 events # 컬렉션 지정
```

- 문서 필드: `customerId` (5,000 가지) · `status` · `amount` · `createdAt` · `region` · `items` · `note`
- 값을 순번에서 계산: 무작위가 아니므로 같은 인자면 모두가 같은 데이터를 봄. `explain()` 출력을 서로 대조할 수 있음
- 지우지 않음: 두 번 돌리면 문서가 그만큼 늘어남. 지우고 넣으려면 `SEED_DROP=1`

적재를 마치며 `fsync` 를 한 번 실행합니다.

WiredTiger 는 60초마다 디스크에 확정하므로, 적재 직후 바로 조회하면 `storageSize` 가 4096 으로 나옵니다. 스크립트가 그 구간을 넘겨 줍니다.

저장 구조 확인

- 1 `MONGO_HOST=10.0.1.5 ~/loadgen/seed-mongo.sh 200000` 으로 `lab.orders` 에 20만 건 적재
- 2 `mongosh "mongodb://10.0.1.5:27017/lab"` 로 접속. 적재 대상이 **lab** 데이터베이스
- 3 `db.orders.stats()` 로 `count` · `size` · `storageSize` · `totalIndexSize` 확인 후 기록
- 4 `size` 와 `storageSize` 의 비로 압축률 계산
- 5 `db.orders.getIndexes()` 로 인덱스가 별도로 존재하는 것 확인

적재 스크립트가 디스크 확정을 한 번 앞당겨 실행합니다.

WiredTiger 는 쌓인 변경을 60초마다 디스크에 확정하므로, 적재 직후 바로 조회하면 `storageSize` 가 4096 으로 나옵니다. `seed-mongo.sh` 가 적재를 마치며 `fsync` 를 한 번 실행해 그 구간을 넘깁니다.

stats() 출력

```
ns: 'lab.orders', count: 200000, avgObjSize: 186, nindexes: 1
size: 37226836, storageSize: 10727424, totalIndexSize: 1998848
```

- **size** : 압축을 풀었을 때의 논리 크기
- **storageSize** : 파일이 디스크에서 차지하는 크기
- **totalIndexSize** : 컬렉션 크기와 별개로 캐시를 씀

storageSize 가 **size** 보다 클 수도 있습니다.

파일에는 재사용을 기다리는 빈 공간이 함께 잡힙니다. 적재량이 적거나 삭제를 섞으면 이렇게 나옵니다.

04

동시성 제어

MVCC

스냅샷

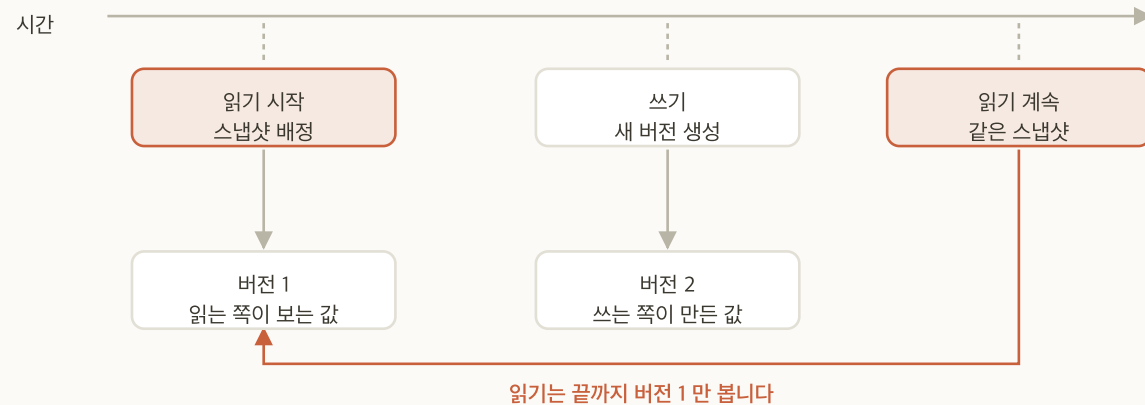
문서 단위

실행 슬롯

쓰기 충돌

재시도

MVCC와 스냅샷



- **MVCC**: Multi-Version Concurrency Control. 값을 덮어쓰지 않고 버전을 새로 만듦
- **스냅샷**: 시작 시점의 상태를 배정받고 끝날 때까지 그것만 봄
- **읽기와 쓰기의 독립**: 읽는 쪽은 옛 버전을, 쓰는 쪽은 새 버전을 봄
- **버전의 자리**: 캐시에 두되 필요하면 history store 파일 (WiredTigerHS.wt)로 내림

읽기 잠금이 없다는 것이 MVCC 의 결과입니다.
조회가 오래 걸려도 그동안의 쓰기가 대기하지 않습니다.

문서 수준 동시성

- **동시성 제어 단위:** 문서 하나. 컬렉션이나 데이터베이스 전체가 아님
- **동시 진행의 범위:** 서로 다른 문서를 고치는 쓰기는 같은 컬렉션이어도 동시에 진행됨
- **병목이 되는 자리:** 같은 문서에 쓰기가 몰릴 때
- **설계로 푸는 방법:** 카운터를 한 문서에 몰지 않고 나눈 뒤 합산

제어 단위가 문서인 것은 MVCC 덕분입니다.

버전을 새로 만드는 구조라 읽는 쪽과 쓰는 쪽이 서로를 기다리지 않습니다.

쓰기 충돌과 재시도

- **동시 실행 슬롯**: WiredTiger 안에서 **동시에 실행될 수 있는 연산 수의 상한**. 읽기용과 쓰기용이 따로 있고, 엔진이 감당하지 못할 만큼 몰리지 않게 입구에서 수를 제한하는 장치
- **얻고 놓는 시점**: 연산이 엔진 작업을 시작할 때 하나 얻고 끝날 때 놓음. 얻지 못하면 큐에서 기다리고, 기다리는 연산은 **out** 에 잡히지 않음
- **낙관적 동시성 제어**: 먼저 잠그지 않고 진행한 뒤 끝낼 때 겹쳤으면 되돌림. **같은 문서**를 동시에 고칠 때만 **WriteConflict** 가 남
- **슬롯 수의 결정**: MongoDB 8 이 CPU 수와 부하를 보며 **실행 중에 조정**. 고정값이 아님
- **서버의 자동 재시도**: 단일 문서 쓰기는 **mongod** 가 스스로 재시도해 애플리케이션에 오류가 가지 않음

지표	읽는 법
<code>queues.execution</code>	<code>out</code> 은 실행 중인 수, <code>available</code> 은 남은 슬롯. 신호는 <code>available</code> 이 0 에 붙는 것
<code>metrics.operation.writeConflicts</code>	충돌 누계. 기동 이후 값이라 증가분을 봄

out 은 동시에 도착한 연산이 여럿일 때만 오릅니다. 연결 하나가 순차로 보내면 1 을 넘지 않습니다. 둘 다 `db.serverStatus()` 아래에 있고, `queues.execution` 은 8.0 에서 `wiredTiger.concurrentTransactions` 가 바뀐 이름입니다. 여러 문서를 묶는 트랜잭션의 재시도는 애플리케이션이 맡습니다.

contend-mongo.sh 경합 스크립트

```
# ops 노드에서. 부하를 주는 쪽과 받는 쪽을 나눕니다
MONGO_HOST=10.0.1.5 ~/loadgen/contend-mongo.sh 8 60 # 8개로 60초
```

인자	기본값	뜻
1번째	8	동시에 도는 프로세스 수. 각자 연결 하나를 잡고 같은 문서를 고침
2번째	60	지속 시간(초). 건수가 아니라 시간으로 끊음

- **경합 대상:** loadtest.hot 의 _id:1 문서 하나. 프로세스 전부가 그 하나를 \$inc
- **문서 하나인 이유:** 낙관적 동시성 제어는 같은 문서를 동시에 고칠 때만 충돌을 냄. 서로 다른 문서를 넣는 부하는 아무리 보내도 충돌이 0
- **출력:** 갱신 횟수와 초당 횟수, 부하 전후의 충돌 누계, 갱신 100회당 충돌

같은 문서 경합 만들기

- 1 `mongosh "mongodb://10.0.1.5:27017/lab"` 로 접속해 부하 전 값을 기록. `db.serverStatus().queues.execution` 과 `db.serverStatus().metrics.operation.writeConflicts`
- 2 `MONGO_HOST=10.0.1.5 ~/loadgen/contend-mongo.sh` 로 60초 부하
- 3 도는 동안 Grafana 에서 쓰기 충돌 증가율·쓰기 슬롯 사용 중·슬롯 잔량·전체 슬롯 수 확인
- 4 끝난 뒤 1번의 두 값을 다시 읽어 대비. 스크립트가 찍은 충돌 증가분과 대조
- 5 `mongosh "mongodb://10.0.1.5:27017/loadtest" --eval 'db.dropDatabase()'` 로 정리

dropDatabase 는 되돌릴 수 없습니다.

`loadtest` 는 이 실습에서만 쓰는 데이터베이스이고 `lab` 은 건드리지 않습니다. 데이터베이스 이름을 확인하고 실행하세요.

05

WiredTiger 캐시와 지속성

캐시 구조

dirty와 clean

eviction

저널

체크포인트

엔진 지표

WiredTiger 캐시

물리 메모리 16 GiB

OS 와 exporter · 약 1 GiB

WiredTiger 캐시
약 7.5 GiB

캐시 밖 사용 · 약 2 GiB

파일시스템 캐시 여유 · 약 5.5 GiB
압축된 채로 남아 재읽기를 줄임

- **캐시에 담기는 대상:** 최근에 접근한 데이터 페이지와 인덱스 페이지
- **기본 계산식:** $(\text{물리 메모리} - 1 \text{ GiB}) \times 0.5$
- **캐시 안의 상태:** 컬렉션 데이터는 압축이 풀려 있고 인덱스는 접두어 압축을 유지
- **인덱스의 자리:** 같은 캐시를 쓰므로 인덱스를 늘리면 데이터가 쓸 캐시가 좁아짐

working set 이 캐시에 들어가는지가 성능을 가릅니다.

캐시를 넘어서면 랜덤 디스크 IO 로 내려가, 성능이 완만하게 나빠지지 않고 계단식으로 떨어집니다.

dirty와 clean

- **페이지의 두 상태:** 캐시에 올라온 페이지는 반드시 **clean** 이거나 **dirty**. 캐시 사용량은 이 둘의 합
- **clean 페이지:** 디스크의 내용과 같은 페이지. 버려도 잃을 것이 없음
- **dirty 페이지:** 갱신되어 디스크와 달라진 페이지. 버리려면 먼저 디스크에 써야 함
- **쓰기가 끝나는 지점:** 쓰기 요청은 dirty 페이지를 만든 시점에 응답을 받음. 디스크 기록은 그 뒤
- **dirty 누적 시의 영향:** 버릴 수 있는 페이지가 줄어 캐시를 비우기 어려워짐
- **확인 방법:** `db.serverStatus().wiredTiger.cache`. 캐시 사용량은 `bytes currently in the cache`, 그중 dirty 는 `tracked dirty bytes in the cache`

캐시가 찼다는 것과 dirty 가 많다는 것은 다른 문제입니다.

clean 이 많으면 즉시 비울 수 있지만, dirty 가 많으면 디스크 쓰기를 기다려야 합니다.

eviction

- **정의:** 캐시에서 페이지를 골라 내보내 자리를 만드는 동작. 평소에는 전용 스레드가 백그라운드에서 수행
- **비용 차이:** clean 은 메모리에서 지우면 끝이고, **dirty 는 디스크에 쓴 뒤 버림**
- **임계값이 걸리는 대상:** 아래 넷은 전부 두 비율에 걸림. 두 비율은 캐시 사용률(`bytes currently in the cache ÷ maximum bytes configured`)과 dirty 비율(`tracked dirty bytes in the cache ÷ 같은 값`)이며, 지금 값은 `db.serverStatus().wiredTiger.cache` 로 읽음

기준	무엇 ÷ 캐시 상한	기본값	넘으면
<code>eviction_target</code>	<code>bytes currently in the cache</code>	80%	백그라운드 eviction 시작
<code>eviction_trigger</code>	<code>bytes currently in the cache</code>	95%	애플리케이션 스레드가 동원됨
<code>eviction_dirty_target</code>	<code>tracked dirty bytes in the cache</code>	5%	dirty 페이지 정리 시작
<code>eviction_dirty_trigger</code>	<code>tracked dirty bytes in the cache</code>	20%	같은 방식으로 동원됨

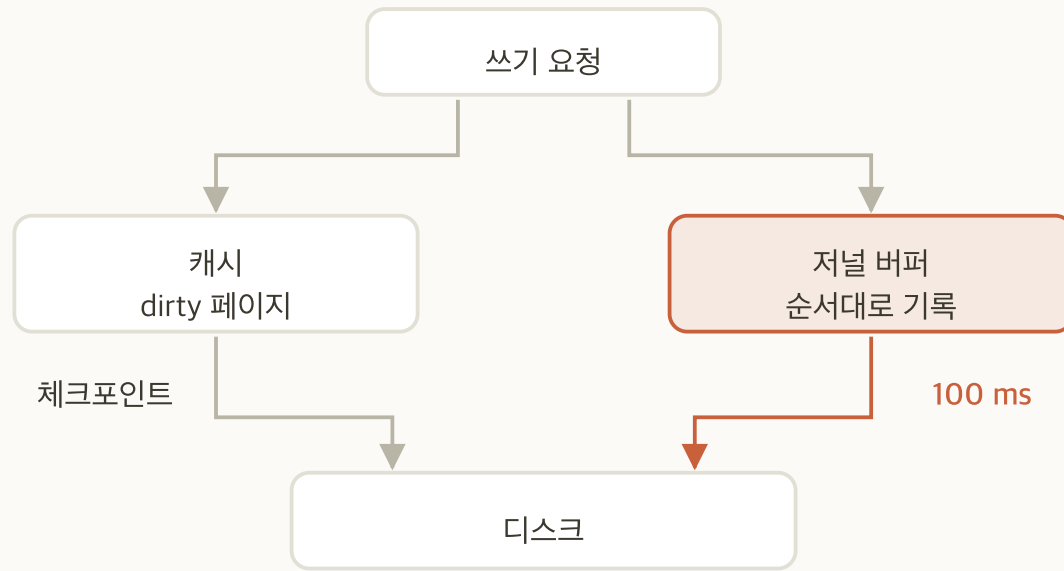
eviction이 지연으로 드러나는 조건

- **평소:** eviction 스레드가 백그라운드로 처리하므로 요청 응답에 드러나지 않음
- **임계값 초과 시:** 요청을 처리하던 스레드가 그 자리에서 eviction 을 직접 수행
- **응답 시간에 미치는 영향:** 그 요청의 응답 시간에 페이지를 디스크에 쓰는 비용이 그대로 더해짐
- **관측되는 양상:** 평균은 정상인데 상위 백분위만 크게 늘고, 캐시 사용률이 임계값 부근에 붙어 있음
- **확인 방법:** `db.serverStatus().wiredTiger.cache` 의 `page evict attempts by application threads` . 지연 누적은 `application thread time evicting (usecs)` 가 더 직접적인 지표

`page evict attempts by application threads` 가 올라가면 캐시가 부족한 것입니다.

이 값이 0 에서 벗어나 계속 증가하면 인스턴스를 키우거나 working set 을 줄일 시점입니다.

저널



- **정의:** 체크포인트로 확정되지 않은 변경을 순서대로 적어 두는 선행 기록
- **필요한 이유:** 캐시의 dirty 페이지는 프로세스가 정지하면 사라짐. 같은 변경을 저널에도 남겨 두었다가 재기동할 때 재생해 그 페이지를 다시 만듦
- **디스크로 내려가는 시점:** 100 밀리초마다. `j:true` 인 쓰기는 저널이 디스크에 닿은 것을 확인한 뒤 응답
- **그룹 커밋:** 같은 시점의 여러 쓰기를 한 번의 디스크 쓰기로 묶음

저널에 기록되기 전의 쓰기는 프로세스가 정지하면 사라집니다.

손실 구간의 크기를 정하는 것이 저널 주기와 Write Concern 의 `j` 옵션입니다.

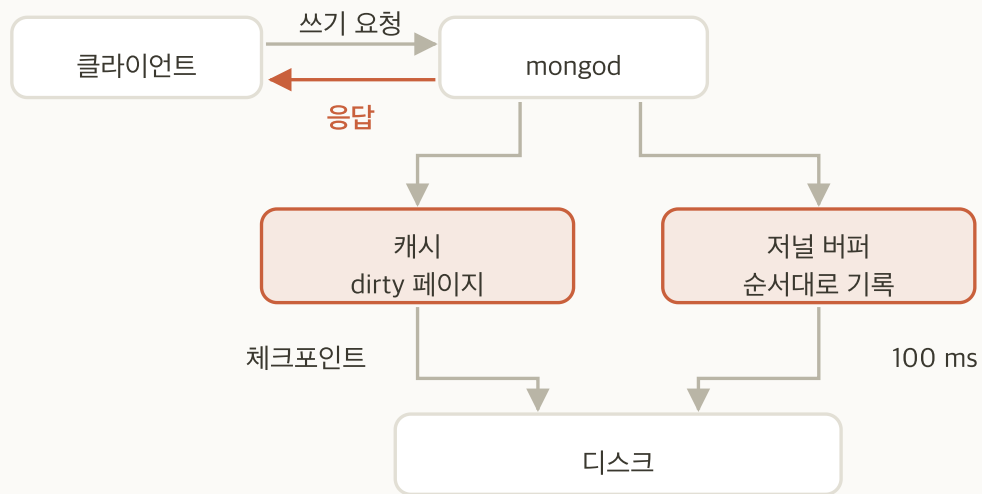
체크포인트

- **정의:** 그 시점의 일관된 상태를 디스크에 확정해 두는 동작
- **실행 시점:** 60초마다. 그 밖에는 `fsync` 나 정상 종료처럼 명시적으로 요구할 때
- **실행 방식:** 스냅샷을 만들고 dirty 페이지를 전부 디스크에 씴. 갱신이 루트까지 전파돼 있으므로 마지막에 루트가 새 위치를 가리키도록 바꾸면 확정
- **끝난 뒤:** 그 이전의 저널은 필요하지 않으므로 정리됨

저널이 체크포인트 사이의 공백을 메웁니다.

재기동하면 마지막 체크포인트를 읽고 그 이후의 저널을 재생합니다.

쓰기 경로

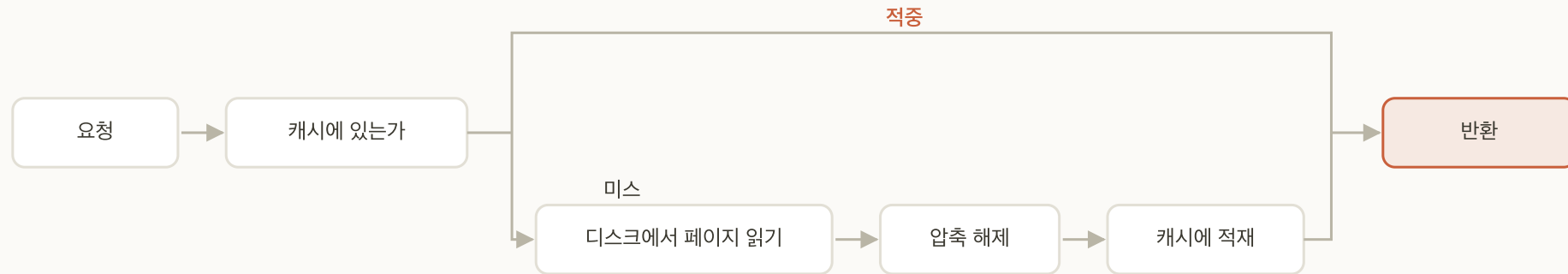


- **응답 시점의 상태:** 디스크가 아니라 위 그림에서 강조한 두 곳, 곧 캐시의 dirty 페이지와 저널 버퍼까지 기록된 상태
- **Write Concern 의 `w`:** 몇 대의 멤버가 받았는지를 셈. `1` · `majority` · 숫자를 넣음
- **Write Concern 의 `j`:** `w` 가 요구하는 멤버 전부에서 저널이 디스크에 닿았는지를 봄. `true` 면 100 밀리초 주기를 기다리지 않고 그 자리에서 내보냄

응답이 돌아온 시점에 데이터가 디스크에 있는 것이 아닙니다.

캐시와 저널 버퍼까지만 기록된 상태입니다. 응답과 디스크 기록 사이의 간격을 얼마로 둘지가 운영 판단입니다.

읽기 경로



- **캐시 적중**: 메모리에서 바로 돌려줌
- **캐시 미스**: 페이지를 디스크에서 읽고 압축을 풀어 캐시에 올린 뒤 돌려줌
- **적재 자리 부족 시**: eviction 이 먼저 일어나야 함

pages read into cache 가 꾸준히 오르면 **working set** 이 캐시를 넘어선 것입니다.

같은 데이터를 반복해서 디스크에서 다시 읽고 있다는 뜻입니다.

엔진 지표

<code>cache.maximum bytes configured</code>	캐시 상한
<code>cache.bytes currently in the cache</code>	지금 캐시에 들어 있는 양
<code>cache.tracked dirty bytes in the cache</code>	그중 dirty 인 양
<code>cache.pages read into cache</code>	디스크에서 캐시로 올린 페이지 수의 누계
<code>cache.page evict attempts by application threads</code>	요청 처리 스레드가 직접 시도한 eviction 수의 누계

전부 `db.serverStatus().wiredTiger` 아래에 있습니다.
누계 카운터는 두 시점의 차이로 봅니다. 절대값은 프로세스 기동 이후의 합입니다.

캐시 지표 확인

- 1 `cd ~/ansible` 후 `ansible mongo-1 -a "free -b"` 로 **mongo-1** 의 물리 메모리 확인. ops 노드의 값이 아님
- 2 `mongosh "mongodb://10.0.1.5:27017/lab"` 로 접속해 `db.serverStatus().wiredTiger.cache["maximum bytes configured"]` 확인. **(물리 메모리 - 1 GiB) × 0.5** 와 대조
- 3 `ansible mongo-1 -m systemd -a "name=mongod state=restarted" --become` 로 재기동해 캐시를 비움
- 4 재접속 후 `db.serverStatus().wiredTiger.cache` 에서 `bytes currently in the cache` 와 `tracked dirty bytes in the cache` 를 기준값으로 기록
- 5 `db.orders.find().itcount()` 로 20만 건을 전부 조회
- 6 4번을 다시 실행해 **두 값의 변화 확인**. 캐시 사용률과 dirty 비율은 **둘 다 상한으로 나눕니다**. 계산해서 임계값 80%·5% 와 대조

지금 값이 이 환경의 기준선입니다.

캐시가 얼마나 차 있고 dirty 가 몇 퍼센트인지는 데이터와 부하마다 다릅니다. 부하가 없는 지금 값을 알아야 나중에 이상을 판별할 수 있습니다.

캐시 지표

db.serverStatus().wiredTiger.cache 의 발췌 · 왼쪽이 재기동 직후, 오른쪽이 20만 건 조회 뒤

```
'maximum bytes configured'      : 7845445632 → 7845445632
'bytes currently in the cache'   :      119627 →      43768359
'tracked dirty bytes in the cache':      49276 →      49276
```

- **캐시 사용률**: 사용량 ÷ 상한. 조회 뒤가 $43768359 \div 7845445632 = 0.56\%$ 로 임계값 80% 에서 멀
- **dirty 비율**: dirty ÷ 상한. 사용량이 아님. 조회 뒤가 0.0006% 로 임계값 5% 에서 멀
- **dirty 증가 없음**: 읽기만 했기 때문. dirty 를 만드는 것은 쓰기뿐

지금 값이 이 환경의 기준선입니다.

캐시 7.31 GiB 에 데이터가 41.7 MiB 라 두 비율 모두 임계값과 거리가 멉니다. 이 여유가 얼마나 줄었는지가 나중에 판단 재료가 됩니다.

16 GiB 노드가 보고하는 물리 메모리는 15.61 GiB 라 상한이 $(15.61 - 1) \times 0.5 = 7.31$ GiB 입니다. 재기동 직후 값은 기동하면서 읽어 올린 카탈로그와 인덱스 메타데이터이고 회차마다 다르므로, 절대값이 아니라 증가분을 봅니다.

06

Replica Set과 선출

멤버 역할

oplog

과반

선출 조건

Rollback

Replica Set의 정의

- **구성**: 같은 데이터를 복제해 두는 `mongod` 여러 대가 하나로 동작
- **쓰기**: PRIMARY 한 대만 받고, 나머지는 그 기록을 따라 적용
- **한 대 정지 시**: 남은 멤버가 같은 데이터를 갖고 있어 서비스가 이어짐
- **담을 수 있는 양**: 데이터를 나눠 담지 않으므로 노드 한 대분 그대로

복제는 가용성을 위한 것이지 용량을 위한 것이 아닙니다.

용량을 늘리려면 Sharding 으로 갑니다. 챕터 12에서 이 구성을 그대로 Sharding 으로 전환합니다.

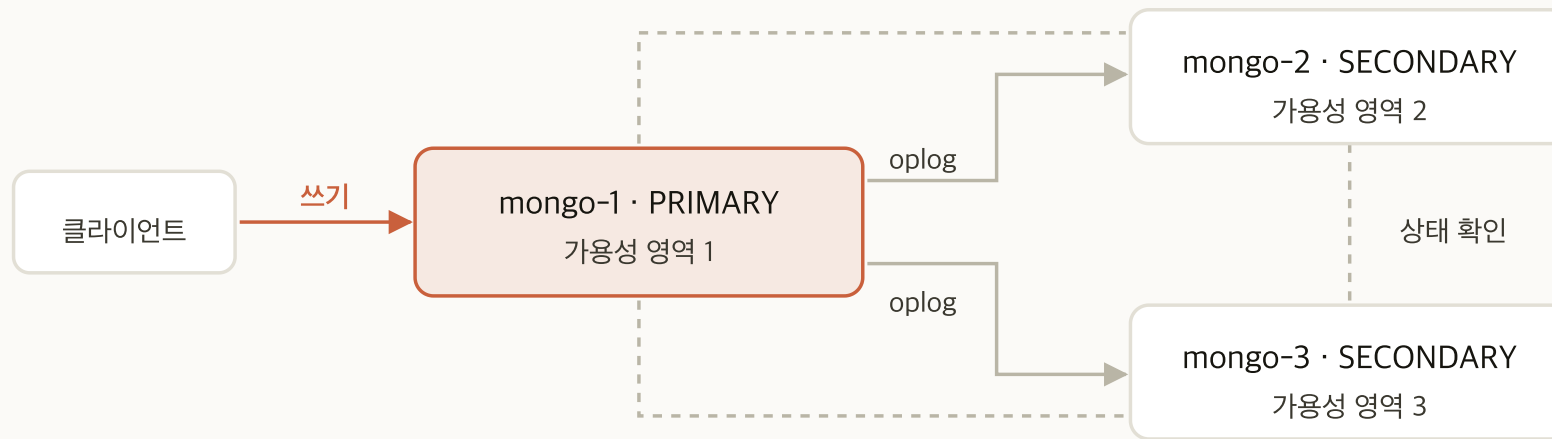
멤버 역할

역할	수행 작업	데이터 보유
PRIMARY	쓰기를 받는 유일한 멤버. 읽기도 받음	예
SECONDARY	oplog 를 재생해 PRIMARY 를 따라감	예
Arbiter	투표만 함	아니오

이 구성에서는 Arbiter 를 쓰지 않습니다.

데이터를 두지 않아 `w:majority` 를 만족시키지 못합니다. 홀수가 필요하면 데이터 멤버를 늘립니다.

3멤버 구성



실선은 데이터 흐름, 점선은 세 멤버가 서로의 상태를 확인하는 경로입니다

- 쓰기는 PRIMARY 하나가 받음
- SECONDARY 는 oplog 를 읽어 같은 변경을 자기 데이터에 적용
- 세 멤버가 서로의 상태를 주기적으로 확인

영역을 나누어 배치하는 이유는 한 영역이 통째로 정지해도 과반이 남기 때문입니다.

oplog

- **정의:** 크기가 고정된 컬렉션. 모든 쓰기를 여러 번 적용해도 결과가 같은 형태로 기록
- **고정 크기인 이유:** 오래된 것부터 덮어씀. 무한히 쌓이지 않음
- **oplog 윈도우:** 가장 오래된 기록부터 최신까지의 시간 폭

oplog 윈도우가 복구 여유입니다.

SECONDARY 가 그보다 오래 멈춰 있으면 따라잡지 못하고 처음부터 다시 동기화합니다.

SECONDARY 정지 시간과 복구 방식

- **정지 시간:** SECONDARY 가 멈춰 있던 동안 PRIMARY 에 쌓인 쓰기를 시간으로 본 것
- **판단 기준:** 그 시간이 oplog 윈도우 안에 들어오는지가 복구 방식을 가름

정지 시간	결과	걸리는 시간
oplog 윈도우 안	oplog 를 재생해 따라잡음	밀린 양만큼
oplog 윈도우 초과	초기 동기화를 처음부터 다시 수행	데이터 전체 크기만큼

- 윈도우가 24시간이면 하루 안에 복구한 노드는 재생으로 따라잡음
- 사흘 뒤에 되살린 노드는 전체를 다시 받음

복제 지연

- **정의:** SECONDARY 가 oplog 를 재생하는 데 걸리는 시간차
- **커지는 시점:** 쓰기가 몰릴 때, SECONDARY 의 디스크가 느릴 때
- **운영상의 의미:** 읽기를 SECONDARY 로 보내면 그만큼 오래된 데이터를 봄

선출의 정의

- **절차:** PRIMARY 가 없어지면 남은 멤버끼리 투표해 새 PRIMARY 를 정함
- **감시 주체:** 별도 감시자 없이 멤버끼리 서로의 상태를 확인
- **선출이 끝날 때까지:** 쓰기는 거부됨

자동 전환이지 무중단은 아닙니다.

선출이 끝날 때까지의 쓰기는 실패합니다. 그 시간을 얼마까지 허용할 것인지가 운영 판단입니다.

멤버 수와 과반

멤버 수	과반	견딜 수 있는 장애	장애 후 선출
2	2	0대	불가
3	2	1대	가능
4	3	1대	가능 (3멤버와 동일)
5	3	2대	가능

짝수는 이득이 없습니다.

4멤버의 과반도 3이므로 견딜 수 있는 장애 수가 3멤버와 같습니다. 비용만 늘어납니다.

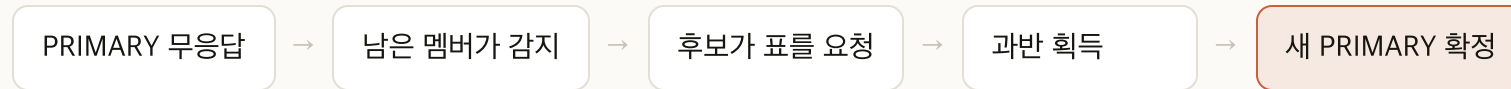
선출이 시작되는 조건

조건	발생 시점
PRIMARY 무응답	<code>electionTimeoutMillis</code> (기본 10초) 동안 응답이 없을 때
수동 강등	<code>rs.stepDown()</code> 을 실행했을 때
우선순위 변경	<code>priority</code> 가 더 높은 멤버가 올라왔을 때

짧게 잡으면 오탐이 늘고, 길게 잡으면 중단이 길어집니다.

일시적 지연을 장애로 오인하는 쪽과 장애 지속 시간이 늘어나는 쪽을 맞바꿉니다.

선출 중의 동작



- 선출 중의 쓰기: 거부됨. `not primary` 계열 오류가 남
- 읽기는 대상 노드를 어떻게 지정했는지에 따라 계속될 수 있음

애플리케이션의 재시도 설정이 체감을 좌우합니다.

드라이버는 기본적으로 재시도 가능한 쓰기를 지원합니다. 켜져 있는지 확인해야 합니다.

Rollback

- **정의:** 정지했던 옛 PRIMARY 가 복귀할 때 그 노드에만 있던 쓰기가 되돌려지는 것
- **생기는 이유:** 그 쓰기를 다른 멤버가 받지 못한 채 PRIMARY 가 정지했기 때문
- **남는 곳:** 데이터 디렉터리에 rollback 파일로 남음

w:majority 가 이것을 막습니다.

과반이 받은 것만 성공으로 돌려주면 되돌려질 쓰기가 애초에 생기지 않습니다.

rs.status() 의 주요 필드

필드	확인 기준	상태 값	뜻
stateStr	PRIMARY 1, SECONDARY 2	PRIMARY	쓰기를 받는 중. 항상 한 대
health	전부 1	SECONDARY	정상 복제 중
optimeDate	멤버 간 차이가 작아야 함	STARTUP2	초기 동기화 중
lastHeartbeatRecv	지금과 가까워야 함	RECOVERING	oplog 를 따라잡는 중
		STARTUP	설정을 읽는 중. 합류 전

(not reachable/healthy) 는 상태 값이 아닙니다.

응답이 없다는 표시입니다. RECOVERING 에 오래 머무르면 oplog 윈도우를 넘긴 것입니다.

단독 노드를 Replica Set으로 전환

- 1 `ansible-playbook play-mongodb.yml -e mongodb_replset_enabled=true`
- 2 `ansible mongo -m systemd -a "name=mongod state=restarted" --become`
- 3 데이터를 넣어 둔 `mongo-1` 에 `mongosh --host 10.0.1.5` 로 접속
- 4 세 멤버 구성 문서를 통째로 넘겨 초기화
- 5 `rs.status()` · `rs.conf()` · `rs.printSecondaryReplicationInfo()` 확인
- 6 `db.getReplicationInfo()` 로 oplog 윈도우 확인하고 **값을 기록**

```
rs.initiate({_id: "rs0", members: [
  {_id: 0, host: "10.0.1.5:27017"},
  {_id: 1, host: "10.0.1.6:27017"},
  {_id: 2, host: "10.0.1.7:27017"}
]})
```

구성 순서의 근거

- **운영에서의 순서:** 설치 시점에 `/etc/mongod.conf` 의 `replication.replSetName` 을 넣고 기동
- **이 세션에서 나눈 이유:** 챕터 03·05의 확인 실습이 초기화 전에 옴. `replication.replSetName` 이 들어간 `mongod` 는 `rs.initiate()` 전까지 쓰기도 조회도 받지 않음
- **부수 효과:** 데이터가 있는 상태에서 초기화하므로 초기 동기화가 실제로 도는 것을 봄
- **이 실습에서 넣는 방법:** 템플릿 `mongod.conf.j2` 가 `mongodb_replset_enabled` 로 `replication` 절을 감싸고 있어, 플레이북에 그 변수를 켜면 `/etc/mongod.conf` 에 두 줄이 더해지고 `mongod` 가 재시작됨

replication 절이 들어가면 그 노드는 세트 구성이 들어올 때까지 대기 상태가 됩니다.
구성이 들어오기 전까지 PRIMARY 가 없어 사용자 데이터베이스에 접근할 수 없습니다.

rs.status() 발췌

```
set: 'rs0',
members: [
  { _id: 0, name: '10.0.1.5:27017', health: 1, stateStr: 'PRIMARY' },
  { _id: 1, name: '10.0.1.6:27017', health: 1, stateStr: 'SECONDARY' },
  { _id: 2, name: '10.0.1.7:27017', health: 1, stateStr: 'SECONDARY' }
]
```

초기화 직후에는 PRIMARY 가 없습니다.

바로 조회하면 SECONDARY STARTUP STARTUP 이 나옵니다. 선출과 초기 동기화가 끝나야 위 상태가 됩니다. 20만 건 기준으로 5초 안에 끝났고, 데이터가 많으면 그만큼 길어집니다.

초기 동기화 확인

- 1 `rs.status()` 를 반복해 두 SECONDARY 가 `STARTUP2` 에서 `SECONDARY` 로 바뀌는 것 확인
 - 2 `mongosh "mongodb://10.0.1.6:27017/lab?readPreference=secondaryPreferred"` 로 SECONDARY 접속
 - 3 `db.orders.countDocuments()` 실행. 챕터 03에서 적재한 건수와 같은지 확인
 - 4 `rs.printSecondaryReplicationInfo()` 와 `db.getReplicationInfo()` 로 지연과 oplog 윈도우 확인
- `rs.add()` 로 나중에 넣은 멤버는 초기 동기화가 끝날 때까지 투표하지 않음
 - 데이터가 클수록 오래 걸림. 운영에서 멤버를 늘릴 때는 PRIMARY 부하를 고려해 시점을 고름

oplog 윈도우 값을 적어 두세요.

챕터 11에서 정지시킨 노드가 재생으로 따라잡을 수 있는지 판단하는 근거입니다.

07

일관성 제어

Write Concern

Read Preference

Read Concern

저널

대가

일관성 제어의 세 축

- 쓰기: 몇 대가 받아야 성공으로 돌려줄 것인가
- 읽기: 어느 멤버에게서 읽을 것인가
- 읽기 기준: 어디까지 확정된 것을 읽을 것인가

안전할수록 느립니다.

기본값을 정하고 예외만 따로 관리하는 것이 운영에서 다루는 방식입니다.

Write Concern

쓰기를 성공으로 돌려주기 전에 몇 대의 멤버가 그 쓰기를 받아야 하는지를 지정하는 값입니다. 요청마다 다르게 줄 수 있습니다.

설정	보장 범위	대가
<code>w:1</code>	PRIMARY 가 받았음	가장 빠르고 되돌려질 수 있음
<code>w:majority</code>	과반이 받았음	되돌려지지 않음. 왕복이 늘어남
<code>j:true</code>	저널까지 내려갔음	디스크 쓰기를 기다림
<code>wtimeout</code>	이 시간 안에 채우지 못하면 오류	없으면 계속 기다릴 수 있음

`w:majority` 는 기본 설정에서 `j:true` 를 포함합니다.

`writeConcernMajorityJournalDefault` 가 `true` 이므로 과반의 저널 기록까지 확인합니다.

Write Concern 선택 기준

설정	사용 상황
<code>w:majority</code>	결제·주문처럼 한 건도 잃으면 안 되는 쓰기. 대부분의 운영 쓰기. 기본 설정에서 과반의 저널까지 포함
<code>w:1, j:true</code>	복제보다 로컬 지속성이 먼저인 쓰기. PRIMARY 의 저널에만 내려가면 성공
<code>w:1</code>	로그·통계처럼 몇 건 사라져도 되는 쓰기. 메모리에 반영되면 성공

`w:majority` 는 「과반이 받았다」이지 「전부 받았다」가 아닙니다.
3멤버면 2대가 받은 시점에 성공으로 돌아옵니다.

Read Preference

드라이버가 읽기 요청을 어느 멤버에게 보낼지 고르는 규칙입니다. 쓰기는 언제나 PRIMARY 로 가지만 읽기에는 선택지가 있습니다.

설정	읽는 대상	사용 상황
primary	PRIMARY 에서만	기본값. 항상 최신을 볼 때
primaryPreferred	PRIMARY 가 없으면 SECONDARY	선출 중에도 읽기를 이어갈 때
secondary	SECONDARY 에서만	분석 질의로 PRIMARY 부하를 피할 때
secondaryPreferred	SECONDARY 가 없으면 PRIMARY	SECONDARY 로 읽되 끊기지 않아야 할 때
nearest	지연이 가장 낮은 멤버	지역이 흩어져 있을 때

SECONDARY 읽기는 복제 지연만큼 오래된 데이터를 봅니다.
얼마나 오래된 것을 허용할 수 있는지가 이 선택의 기준입니다.

Read Concern

읽어 오는 데이터가 **어디까지** **확정된 것**이어야 하는지를 지정하는 값입니다. Read Preference 가 어디서 읽을지를 정한다면, Read Concern 은 무엇을 읽을지를 정합니다.

설정	보는 범위
local	그 멤버에 적용된 것. 되돌려질 수 있음
available	local 과 같되 Sharding 환경에서 이동 중인 문서까지 봄
majority	과반이 받은 것만. 되돌려지지 않음
linearizable	가장 최근의 확정된 쓰기. PRIMARY 에서만 동작하고 가장 느림

Write Concern 과 짝을 맞춰야 뜻이 생깁니다.

w:1 로 쓰고 **readConcern:majority** 로 읽으면 방금 쓴 것이 보이지 않을 수 있습니다.

조합 구성의 원칙

- 실제로 갈리는 축 셋: 되돌려질 수 있는가는 `w`, 최신을 보는가는 Read Preference, 확정된 것만 보는가는 Read Concern 이 정함

어긋난 조합

문제

`w:1` + `readConcern:majority`

방금 쓴 것이 아직 과반에 없어 자기 쓰기를 읽지 못함

`secondary` + `linearizable`

PRIMARY 에서만 동작하므로 성립하지 않음

쓰기를 강하게 잡았으면 읽기도 그만큼 잡아야 합니다.

한쪽만 올리면 지연은 늘어나는데 보장은 약한 쪽에 맞춰집니다.

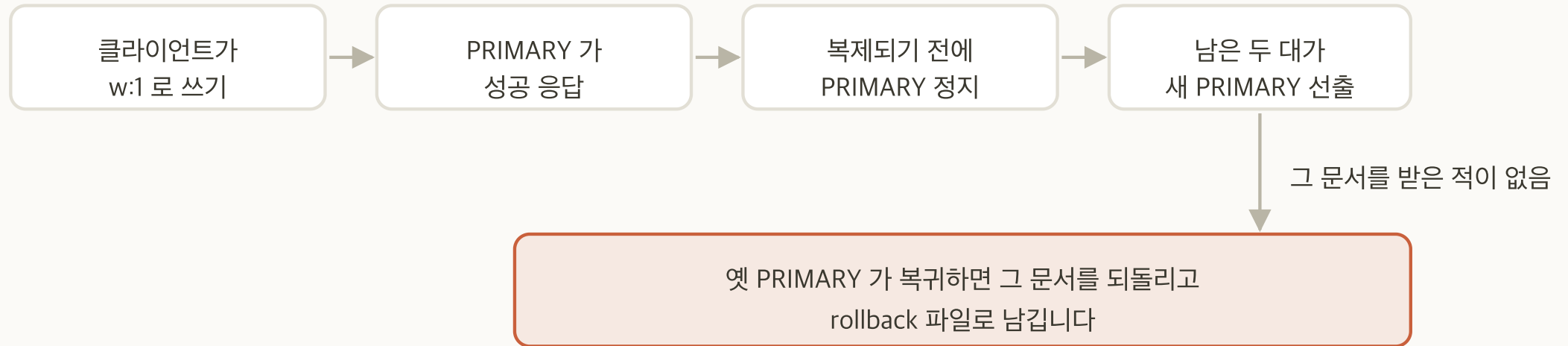
업무 성격별 권장 조합

업무 성격	Write Concern	Read Preference	Read Concern	고른 이유
결제·주문	w:majority	primary	majority	한 건도 잃을 수 없고 방금 쓴 것을 바로 읽어야 함
일반 조회·수정	w:majority	primary	local	기본값. PRIMARY 에서 읽으므로 자기 쓰기는 보이지만, 아직 과반에 없는 것도 함께 보임
분석·리포트	w:1	secondary	local	PRIMARY 부하를 피함. 조금 오래된 데이터를 감수함
로그·통계	w:1	primary	local	몇 건 사라져도 되는 쓰기

기본값을 하나 정하고 예외만 따로 관리합니다.

요청마다 다르게 고르기 시작하면 어느 데이터가 어떤 보장을 받는지 파악할 수 없게 됩니다.

Rollback과 w:majority



w:majority 로 쓰면 이 경로 자체가 생기지 않습니다

- `w:1` 은 PRIMARY 가 받은 시점에 성공을 돌려줌
- 그 문서가 SECONDARY 로 가기 전에 PRIMARY 가 정지하면 사라짐
- `w:majority` 는 과반이 받을 때까지 성공을 돌려주지 않음

w:majority 는 되돌려질 쓰기를 애초에 만들지 않습니다. 과반이 받은 것만 성공으로 돌려주므로 옛 PRIMARY 에만 남은 쓰기가 생기지 않습니다.

저널과 j 옵션

- **정의:** 체크포인트 사이의 쓰기를 순차로 기록하는 로그
- **필요한 이유:** 프로세스가 정지해도 마지막 체크포인트 이후를 재생할 수 있음
- **j:true** 의 뜻: **w** 가 요구하는 **멤버 전부**가 저널에 내려쓴 것을 확인하고 응답. 예전에는 PRIMARY 한 대만 봤음
- **j** 의 기본값은 **w** 가 정함: **w** 가 숫자이면 **j:false** , **w:majority** 이면 **writeConcernMajorityJournalDefault** (기본값 **true**)를 따라 **j:true**

j 의 기본값은 w 가 정합니다. w 가 숫자이면 j:false, w:majority 이면 writeConcernMajorityJournalDefault 를 따라 j:true 입니다. 그래서 w:majority, j:true 는 같은 요구를 두 번 적은 것이고, w:1, j:true 는 없던 요구를 더한 것입니다.

wc-measure.sh 측정 스크립트

```
# ops 노드에서. MONGO_HOST 는 현재 PRIMARY 여야 합니다
MONGO_HOST=<PRIMARY> ~/loadgen/wc-measure.sh # 건수는 WC_COUNT 로
```

재는 조합

추가로 기다리는 것

<code>w:1</code>	없음
<code>w:1, j:true</code>	PRIMARY 의 저널 기록
<code>w:majority</code>	과반의 저널 기록과 복제 왕복

조합마다 기본 1,000건을 한 건씩 넣는 것이 핵심입니다.

`insertMany` 로 넣으면 Write Concern 이 배치 하나에 한 번만 적용되어 세 값이 같아집니다.

세 조합 측정

- 1 `mongosh "mongodb://10.0.1.5:27017/lab" --eval 'rs.status().members.map(m => [m.name, m.stateStr])'` 로 현재 PRIMARY 확인
- 2 `MONGO_HOST=<PRIMARY> ~/loadgen/wc-measure.sh` 실행
- 3 세 조합마다 1,000건을 한 건씩 순차 삽입하고 소요 시간 측정
- 4 출력된 표의 세 값을 기록

한 건씩 넣습니다.

`insertMany` 로 한 번에 넣으면 Write Concern 이 배치 하나에 한 번만 적용되어 세 값이 거의 같아집니다. 스크립트가 이 방식을 고정합니다.

완료 판정

세 값이 `w:1` 보다 `w:1, j:true` 가, 그보다 `w:majority` 가 큰 순서로 나오면 완료입니다.

세 조합의 소요 시간

설정	추가 대기 대상	1,000건 소요
<code>w:1</code>	없음	0.7초
<code>w:1, j:true</code>	PRIMARY의 저널 기록	1.3초
<code>w:majority</code>	과반의 저널 기록과 복제 왕복	2.0초

`w:1` 과 `j:true` 의 차이가 저널 비용, `j:true` 와 `w:majority` 의 차이가 복제 비용입니다.

`w:majority` 가 저널 대기를 포함하므로 순서가 뒤집히지 않습니다.

wc-majority-loss.sh 재현 스크립트

```
# ~/ansible 에서 실행합니다. ansible.cfg 의 inventory 가 상대 경로입니다
cd ~/ansible
MONGO_HOST=<PRIMARY> ~/loadgen/wc-majority-loss.sh
```

- 하는 일: w:majority 쓰기 루프를 띄우고 SECONDARY 를 한 대씩 정지시킨 뒤 되살림
- 노드를 내리는 방법: ansible 로 mongod 를 정지. 그래서 인벤토리가 필요함
- 시점을 손으로 맞추지 않는 이유: 과반이 깨지고 5초에서 8초 만에 강등되므로 놓치기 쉬움

어떻게 끝나든 두 노드를 다시 기동합니다.

두 노드를 내린 채로 끝나면 이후 챕터가 전부 막히므로 스크립트가 복구를 보장합니다. 그래도 끝난 뒤 `rs.status()` 로 PRIMARY 1 SECONDARY 2 를 확인하세요.

과반 상실 시의 쓰기 동작

- 1 `rs.status()` 로 현재 PRIMARY 와 SECONDARY 두 대를 확인
 - 2 `cd ~/ansible` 후 `MONGO_HOST=<PRIMARY> ~/loadgen/wc-majority-loss.sh` 실행. `~/ansible` 밖에서 실행하면 노드를 찾지 못합니다
 - 3 출력에서 두 오류가 순서대로 나왔는지 확인
 - 4 `wtimeout` 으로 실패한 문서가 PRIMARY 에 남아 있는지 조회해서 확인
 - 5 스크립트가 알려 준 현재 PRIMARY 를 기록. 이후 챕터의 `--host` 는 이 노드입니다
- **스크립트가 하는 일:** 쓰기 루프를 띄우고 SECONDARY 를 한 대씩 정지시킨 뒤 되살림
 - **스크립트의 안내:** 단계마다 무엇이 왜 일어나는지 먼저 알리고 결과를 시각과 함께 출력
 - **시점을 손으로 맞추지 않는 이유:** 과반이 깨지고 5초에서 8초 만에 강등되므로 시점을 손으로 맞추면 놓치기 쉬움
 - **되살리기:** 스크립트가 어떻게 끝나든 두 노드를 다시 기동

`~/loadgen/wc-majority-loss.sh` 는 실습 저장소의 과반 상실 재현 스크립트입니다. `MONGO_HOST` 로 대상을 정하고, 노드를 내리는 데 `ansible` 을 씁니다. `ansible.cfg` 의 `inventory` 가 상대 경로라 `~/ansible` 에서 실행해야 합니다. 완료 판정 · 두 오류가 차례로 나왔고 세 멤버가 다시 PRIMARY 1 SECONDARY 2 로 돌아왔으면 완료입니다.

스크립트 출력 발췌

```
15:35:35 -- mongo-2 정지 --
15:35:35 OK
15:35:36 OK
15:35:37 OK
```

```
15:35:46 -- mongo-3 정지 --
15:35:47 WriteConcernTimeout
15:35:51 NotWritablePrimary
15:35:55 NotWritablePrimary
```

오류	PRIMARY 의 상태	뜻
WriteConcernTimeout	아직 PRIMARY	쓰기는 받았지만 과반이 받았는지 확인하지 못함
NotWritablePrimary	SECONDARY 로 강등됨	쓰기를 받을 자격 자체가 없어짐

wtimeout 은 「쓰지 않았다」가 아닙니다.

그 문서는 PRIMARY 에 이미 적용돼 있습니다. 과반이 받았는지 확인하지 못했을 뿐이고, 그래서 나중에 되돌려질 수 있습니다.

08

Index와 실행 계획

인덱스 종류

ESR

실행 계획

covered query

인덱스 비용

인덱스의 정의

- **구조**: 특정 필드의 값을 정렬해 따로 유지하는 자료구조
- **없을 때의 조회**: 컬렉션 전체를 읽어 조건에 맞는 것을 고름
- **쓰기에 미치는 영향**: 쓰기마다 모든 인덱스를 함께 갱신하므로 쓰기가 무거워짐

인덱스는 쓰기 비용을 치르고 읽기 속도를 얻는 교환입니다.
많이 만들수록 좋은 것이 아니라, 쓰이는 만큼만 두는 것이 기준입니다.

인덱스 B-tree

항목	컬렉션 B-tree	인덱스 B-tree
키	RecordId	인덱스에 지정한 필드 값
leaf 의 저장 내용	문서	인덱스 키와 RecordId
개수	컬렉션마다 하나	인덱스마다 하나씩
조회에서의 역할	RecordId 로 문서를 꺼냄	조건에 맞는 RecordId 를 찾음

- 값이 정렬되어 있어 범위 조회와 정렬 요청을 그대로 처리

조회는 두 트리를 탐니다.

인덱스 B-tree 에서 RecordId 를 찾고, 그 번호로 컬렉션 B-tree 에서 문서를 꺼냅니다. 실행 계획의 **IXSCAN** 과 **FETCH** 가 이 두 단계입니다.

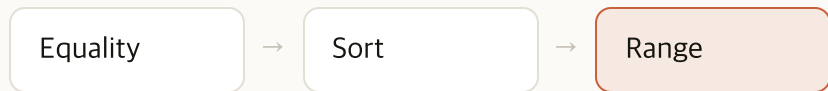
인덱스 종류

종류	내용	사용 상황
단일	필드 하나	그 필드로만 찾을 때
복합	여러 필드를 순서대로	조건이 여럿이거나 정렬이 붙을 때
멀티키	배열 필드	배열 원소로 찾을 때
TTL	시간 필드 기준 자동 삭제	세션·임시 데이터 만료
부분	조건에 맞는 문서만	인덱스 크기를 줄일 때
희소	그 필드가 있는 문서만	필드가 일부 문서에만 있을 때
고유	중복 금지	업무 키의 유일성 보장

부분은 조건으로 고르고 희소는 필드의 존재로 고릅니다. 실습에서 만드는 것은 단일과 복합 둘입니다.

ESR 규칙

복합 인덱스에 필드를 **어떤 순서로 넣을지** 정하는 규칙입니다. 같은 필드 조합이어도 순서가 다르면 인덱스가 쓰이는 방식이 달라집니다.



- **Equality:** `=` 로 값을 고정하는 필드를 앞에 둡니다
- **Sort:** 정렬에 쓰는 필드를 그다음에 둡니다
- **Range:** `$gt` · `$lt` 같은 범위 조건을 마지막에 둡니다

앞 필드가 고정돼야 뒤 필드의 정렬이 남습니다.

인덱스는 앞 필드부터 차례로 정렬돼 있습니다. 앞 필드가 하나의 값으로 고정되면 그 안에서 다음 필드가 정렬된 상태로 남습니다. 범위 조건은 값을 하나로 묶지 못하므로 그 뒤 필드의 정렬이 흩어집니다.

ESR 예시

`status` 가 `refunded` 이고 `amount` 가 90000 이상인 주문을 `createdAt` 내림차순으로 조회할 때

인덱스 순서	탐색 과정	결과
<code>{ status:1, createdAt:-1, amount:1 }</code>	<code>status</code> 가 하나로 고정되고, 그 안에서 <code>createdAt</code> 이 이미 내림차순으로 놓여 있음	인덱스를 순서대로 읽은 것이 곧 정렬 결과. <code>\$SORT</code> 가 없음
<code>{ status:1, amount:1, createdAt:-1 }</code>	<code>amount >= 90000</code> 이 범위라 값이 하나로 묶이지 않음. <code>createdAt</code> 은 <code>amount</code> 값 마다 따로 정렬돼 전체로는 뒤섞임	메모리에서 다시 정렬함. <code>\$SORT</code> 가 붙음

범위 조건 뒤의 필드는 정렬을 잃습니다.

범위는 값을 하나로 고정하지 못하므로, 그 뒤에 놓인 필드는 구간마다 따로 정렬된 상태가 됩니다.

실행 계획

- **탐색 경로:** 컬렉션 전체인지 인덱스인지
- **검사한 양:** 검사한 키와 문서의 수
- **메모리 정렬 여부:** SORT 단계가 있는지

단계	뜻
COLLSCAN	인덱스가 쓰이지 않음
IXSCAN	인덱스가 쓰임
FETCH	RecordId 로 문서를 읽음
SORT	인덱스가 정렬 순서를 제공하지 못함
PROJECTION_COVERED	인덱스만으로 결과를 만듦

단계는 바깥에서 안으로 중첩됩니다.

SORT 가 있으면 그것이 바깥이고 IXSCAN 이 안쪽입니다. 화면에서는 SORT 가 위에 보입니다.

인덱스 품질 측정

지표	뜻	판단
<code>nReturned</code>	실제로 돌려준 문서 수	비교 기준
<code>totalKeysExamined</code>	검사한 인덱스 키 수	<code>nReturned</code> 와 가까울수록 좋음
<code>totalDocsExamined</code>	읽은 문서 수	0이면 covered query
<code>executionTimeMillis</code>	걸린 시간	캐시 상태에 따라 흔들림

검사한 것 대비 돌려준 것의 비율이 인덱스 품질입니다.

`totalKeysExamined ÷ nReturned` 가 1에 가까울수록 낭비 없이 찾은 것입니다.

covered query

- **정의:** 필요한 값이 전부 인덱스에 있어 문서를 읽지 않는 조회
- **조건:** 조회 조건과 반환 필드가 모두 인덱스 안에 있어야 함
- **확인 방법:** `totalDocsExamined` 가 0

문서를 읽지 않으므로 컬렉션 B-tree 를 타지 않습니다.
 조회가 두 트리에서 한 트리로 줄어드는 것이 이 형태의 이득입니다.

SORT 단계의 상한

- 다시 정렬이 생기는 조건: 인덱스가 정렬 순서를 제공하지 못할 때
- 메모리 정렬 상한: 100 MB. 넘으면 디스크에 임시 파일을 써서 정렬을 이어 감
- 상한 초과 시의 동작: 응답이 느려지고 디스크 IO 가 늘어남. `allowDiskUse(false)` 로 막아 둔 경우에만 조회가 실패
- 이 단계를 없애는 방법: 인덱스가 정렬 순서를 그대로 제공하게 만듦

SORT 가 보이면 인덱스 설계를 다시 봅니다.

데이터가 늘면 상한을 넘어 디스크로 흘러 조회 시간이 예측을 벗어납니다.

인덱스 생성 부하

- 인덱스를 만드는 동안 그 컬렉션의 성능이 떨어짐
- 운영에서는 SECONDARY 부터 한 대씩 만드는 롤링 생성을 씀
- 백업에서 복구할 때도 인덱스를 다시 만듦

이 부하가 챕터 09 복구 시간으로 나타납니다.

논리 백업의 복구가 느린 이유는 문서를 넣은 뒤 인덱스를 다시 만들기 때문입니다.

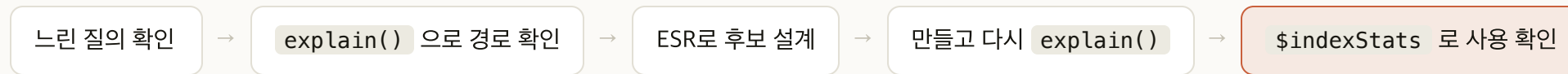
인덱스 남용

- 쓰기마다 모든 인덱스를 갱신함. 인덱스 수만큼 쓰기가 무거워짐
- 인덱스도 캐시를 차지함. 그만큼 데이터가 쓸 캐시가 줄어들음
- **쓰이지 않는 인덱스에는 비용만 남음**

`$indexStats` 로 실제로 쓰이는지 확인합니다.

만든 사람이 기억하는 것과 실제로 쓰이는 것은 다릅니다.

인덱스 설계 순서



질의를 확인한 뒤에 인덱스를 만듭니다.

만들어 두고 쓰이기를 기다리는 인덱스는 쓰기 비용만 늘립니다.

인덱스 없이 조회

```
# ops 노드에서. 현재 PRIMARY 를 확인하고 그 노드에 접속합니다
mongosh "mongodb://10.0.1.5:27017/lab" --eval 'rs.status().members.map(m => [m.name, m.stateStr])'
mongosh "mongodb://<PRIMARY>:27017/lab"
db.orders.find({ status: "refunded", amount: { $gte: 90000 } })
      .sort({ createdAt: -1 }).explain("executionStats")
```

```
winningPlan: { stage: 'SORT', inputStage: { stage: 'COLLSCAN' } }
nReturned: 3335, executionTimeMillis: 53
totalKeysExamined: 0, totalDocsExamined: 200000
```

3,335건이 필요한데 20만 건을 전부 읽었습니다. 이 값이 기준값입니다.

단일 필드 인덱스 적용

```
db.orders.createIndex({ status: 1 })
db.orders.find({ status: "refunded", amount: { $gte: 90000 } })
    .sort({ createdAt: -1 }).hint({ status: 1 }).explain("executionStats")
```

```
{ stage: 'SORT', inputStage: { stage: 'FETCH',
    inputStage: { stage: 'IXSCAN', indexName: 'status_1' } } }
nReturned: 3335, executionTimeMillis: 29
totalKeysExamined: 33333, totalDocsExamined: 33333
```

- 검사한 문서 수: 20만에서 33,333으로 감소. `status` 로만 걸렀기 때문
- `.hint()` 를 붙이는 이유: 인덱스가 쌓이면 선택이 정해지지 않음

ESR 위반 복합 인덱스

```
db.orders.createIndex({ status: 1, amount: 1, createdAt: -1 })
db.orders.find({ status: "refunded", amount: { $gte: 90000 } })
  .sort({ createdAt: -1 }).limit(20)
  .hint({ status: 1, amount: 1, createdAt: -1 }).explain("executionStats")
```

```
winningPlan: { stage: 'FETCH', inputStage: { stage: 'SORT',
  sortPattern: { createdAt: -1 }, inputStage: { stage: 'IXSCAN',
    indexName: 'status_1_amount_1_createdAt_-1' } } }
nReturned: 20, totalKeysExamined: 3335, totalDocsExamined: 20
```

조건은 좁혔지만 정렬 순서를 주지 못해 3,335건을 읽어 정렬했습니다.

ESR 준수 복합 인덱스

```
db.orders.createIndex({ status: 1, createdAt: -1, amount: 1 })
db.orders.find({ status: "refunded", amount: { $gte: 90000 } })
  .sort({ createdAt: -1 }).limit(20)
  .hint({ status: 1, createdAt: -1, amount: 1 }).explain("executionStats")
```

```
winningPlan: { stage: 'LIMIT', inputStage: { stage: 'FETCH',
  inputStage: { stage: 'IXSCAN',
    indexName: 'status_1_createdAt_-1_amount_1' } } }
nReturned: 20, totalKeysExamined: 83, totalDocsExamined: 20
```

SORT 가 사라지고 검사한 키가 3,335에서 83으로 줄었습니다. 인덱스가 정렬 순서를 주므로 20건을 채우는 순간 멈춥니다.

\$indexStats 로 사용 여부 확인

```
// explain() 을 떼고 같은 조회를 실제로 두 번 실행합니다
db.orders.find({ status: "refunded", amount: { $gte: 90000 } })
    .sort({ createdAt: -1 }).limit(20).toArray()
db.orders.aggregate([ { $indexStats: {} } ])
```

```
{ name: '_id_',                accesses: { ops: 0 } }
{ name: 'status_1',            accesses: { ops: 0 } }
{ name: 'status_1_amount_1_createdAt_-1', accesses: { ops: 0 } }
{ name: 'status_1_createdAt_-1_amount_1', accesses: { ops: 2 } }
```

\$indexStats 는 **explain()** 을 세지 않습니다.

앞의 네 단계를 **explain()** 으로만 돌렸다면 전부 **ops: 0** 입니다. 인덱스는 지우지 않습니다. 챕터 09 복구 검증이 이 목록을 씁니다.

09

Backup과 Restore

논리 백업

물리 백업

--oplog

PITR

RTO·RPO

복구 리허설

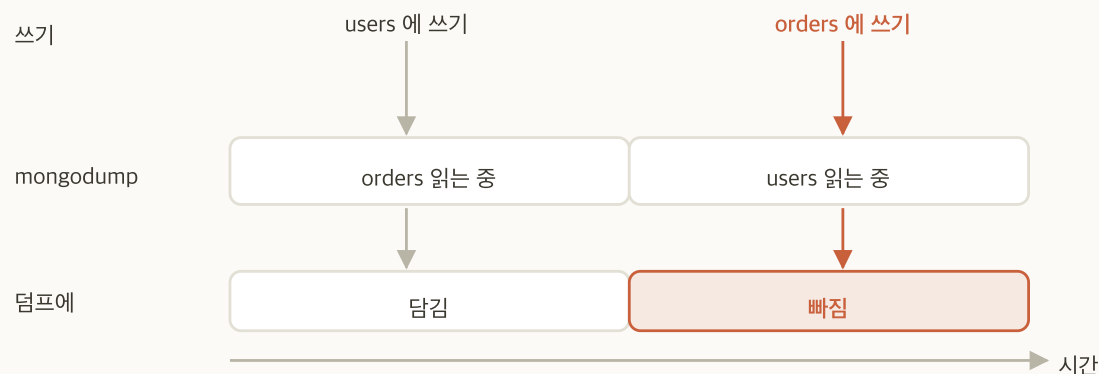
백업 방식 두 갈래

방식	담는 것	대가
논리 백업 (<code>mongodump</code>)	문서를 읽어 BSON 으로	이식성이 좋고, 데이터가 크면 느림
물리 백업 (볼륨 스냅샷)	데이터 파일을 통째로	빠르고, 같은 구성에서만 복구됨

논리 백업의 복구가 느린 이유는 인덱스를 다시 만들기 때문입니다.
인덱스 생성 부하가 그대로 복구 시간이 됩니다.

--oplog 의 필요성

oplog 는 모든 쓰기를 순서대로 적어 두는 고정 크기 컬렉션입니다.

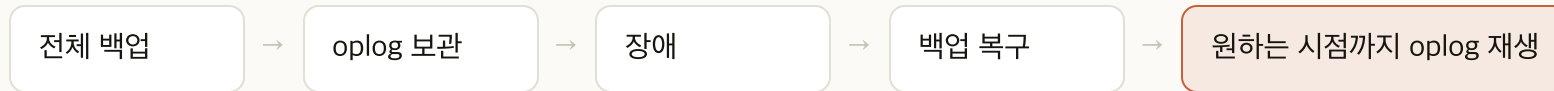


- **읽는 순서:** 덤프가 컬렉션을 하나씩 읽는 동안에도 쓰기가 들어옴. 이미 읽은 컬렉션에 들어온 쓰기는 빠짐
- **--oplog** : 덤프가 도는 구간의 oplog 를 함께 담고, 복구할 때 **--oplogReplay** 로 재생해 한 시점으로 맞춤

--oplog 없이 만든 덤프는 시점이 뒤섞여 있습니다.

이 옵션은 단독 노드에서 동작하지 않습니다. Replica Set 에서만 쓸 수 있습니다.

PITR



- **정의:** Point-In-Time Recovery. 백업 시점이 아니라 **임의의 시점**으로 되돌리는 복구
- **필요한 것:** 앞 장에서 본 `--oplog` 덤프 하나와, 그 뒤로 끊기지 않고 남아 있는 oplog
- **방지 대상:** 잘못된 삭제나 배포 사고처럼 「어제 오후 3시로 돌리고 싶다」는 상황

자체 운영에서는 직접 구성해야 합니다.

관리형 서비스처럼 기능으로 주어지지 않으므로, 백업 주기와 oplog 보관 기간을 직접 맞춰야 성립합니다.

백업 대상 멤버

대상	이점	대가
PRIMARY	가장 최신 시점	덤프 부하가 서비스 요청과 같은 노드에 몰림
SECONDARY	서비스 부하와 분리	복제 지연만큼 오래된 시점을 담음

- **운영의 기본 대상:** SECONDARY. 덤프는 컬렉션 전체를 읽으므로 캐시를 밀어냄
- **SECONDARY 백업 시 확인 항목:** 복제 지연이 작아야 담기는 시점이 최신에 가까움
- **이 실습의 대상:** PRIMARY. 3대뿐이고 부하가 없어 차이가 드러나지 않음

RTO와 RPO

용어	뜻	결정 대상
RTO	복구까지 허용되는 시간	백업 방식과 복구 절차
RPO	잃어도 되는 데이터 구간	백업 주기와 oplog 보관

- **RTO 30분**: 논리 백업 복구가 2시간 걸린다면 물리 스냅샷을 써야 함
- **RPO 5분**: 하루 한 번 덤프로는 맞출 수 없음. oplog 재생이 필요

방식을 고르고 나서 RTO·RPO 를 계산하지 않습니다.
순서가 반대입니다. 무엇을 몇 분 안에 되살려야 하는지가 방식을 정합니다.

복구 리허설

- 복구해 본 적 없는 백업은 백업이 아님
- 덤프가 만들어졌다는 것과 복구된다는 것은 다른 사실
- 복구 소요 시간을 재 두어야 RTO 를 말할 수 있음

이 실습이 복구 리허설 그 자체입니다.

덤프를 만들고, 데이터를 지우고, 되살린 뒤 일치하는지까지 확인합니다.

백업 준비와 기준값 기록

```
db.getCollectionNames().filter(n => n.startsWith("wc_")).forEach(n => db[n].drop())
db.orders.countDocuments()
db.orders.getIndexes().map(i => i.name)
db.orders.findOne({}, { customerId: 1, status: 1 })
```

```
200000
[ '_id_', 'status_1', 'status_1_amount_1_createdAt_1',
  'status_1_createdAt_1_amount_1' ]
{ _id: ObjectId('...'), customerId: 'C000000', status: 'paid' }
```

세 값을 적어 두세요. 복구가 끝난 뒤 같은 세 명령으로 대조합니다.

덤프 실행

```
# ops 노드의 셸에서. mongosh 안이 아닙니다
mongodump --host <PRIMARY> --oplog --out ~/dump
```

```
done dumping `lab.orders` (200000 documents)
writing captured oplog to ``
    dumped 1 oplog entry
```

--oplog 는 Replica Set 에서만 동작합니다.

단독 노드에서 실행하면 이 줄이 나오지 않고 시점이 뒤섞인 덤프가 만들어집니다.

덤프 결과 확인

```
du -sh ~/dump
ls ~/dump ~/dump/lab
```

```
36M   /home/azureuser/dump
~/dump:      admin  lab  oplog.bson  prelude.json
~/dump/lab:  orders.bson  orders.metadata.json
```

덤프가 컬렉션 파일보다 큼니다.

덤프 디렉터리가 36 MB 인데 챕터 03에서 본 `storageSize` 는 10.7 MB 였습니다. 덤프는 압축을 푼 BSON 이므로, 백업 공간을 압축된 크기 기준으로 잡으면 부족합니다.

데이터베이스 삭제

```
// PRIMARY 에 접속해서
use lab
db.dropDatabase()
// 지워진 것 확인
db.orders.countDocuments()
```

- **확인 지점:** `countDocuments()` 가 0 을 돌려주고 `show collections` 가 비어 있음
- **복제 확인:** SECONDARY 에서도 함께 비었는지 확인. 삭제도 복제됨

데이터베이스 삭제는 되돌릴 수 없습니다.

앞 장에서 `oplog.bson` 과 `orders.bson` 이 만들어진 것을 눈으로 확인한 뒤에 실행하세요. 덤프가 없는 상태에서 실행하면 20만 건을 다시 적재해야 합니다.

복구 실행

```
# ops 노드의 셸에서
mongorestore --host <PRIMARY> --oplogReplay ~/dump
```

```
restoring `lab.orders` from `/home/azureuser/dump/lab/orders.bson`
finished restoring `lab.orders` (200000 documents, 0 failures)
replaying oplog
applied 1 oplog entries
restoring indexes for collection `lab.orders` from metadata
200000 document(s) restored successfully. 0 document(s) failed to restore.
```

복구가 덤프보다 오래 걸립니다. 덤프가 0.2초, 복구가 3.3초입니다.

일치 검증

```
// 백업 준비에서 쓴 것과 같은 세 명령
db.orders.countDocuments()
db.orders.getIndexes().map(i => i.name)
db.orders.findOne({}, { customerId: 1, status: 1 })
```

```
200000
[ '_id_', 'status_1_createdAt_-1_amount_1', 'status_1',
  'status_1_amount_1_createdAt_-1' ]
{ _id: ObjectId('...'), customerId: 'C000000', status: 'paid' }
```

세 값이 모두 덤프 이전과 같아야 복구가 끝난 것입니다.

10

권한

인증과 인가

내부 인증

내장 Role

사용자 정의 Role

활성화 순서

롤링 재시작

인증과 인가

갈래	수단	답하는 질문
인증	SCRAM-SHA-256, x.509	누구인가
인가	Role	무엇을 할 수 있는가

둘은 다른 질문입니다.
 들어올 수 있다는 것과 무엇이든 할 수 있다는 것은 같지 않습니다.

내장 Role

Role	권한 범위	주는 대상
<code>read</code>	읽기만	조회 전용 계정
<code>readWrite</code>	읽기와 쓰기	애플리케이션 계정
<code>dbAdmin</code>	인덱스·통계 관리	운영자
<code>userAdmin</code>	계정과 Role 관리	관리자
<code>clusterMonitor</code>	상태 지표 조회만	exporter 계정
<code>clusterAdmin</code>	<code>rs.status()</code> 등 클러스터 조작과 조회	운영 관리자
<code>root</code>	전부	일반 계정에는 부여하지 않음

Role 은 데이터베이스 단위로 부여됩니다. 이름 끝에 AnyDatabase 가 붙은 것(`userAdminAnyDatabase`·`readWriteAnyDatabase`)은 모든 데이터베이스에 걸리며 `admin` 에서만 부여합니다. `clusterAdmin`·`clusterMonitor` 도 같습니다.

사용자 정의 Role

- **정의:** 액션 단위로 골라 조합한 Role
- **사용 상황:** 내장 Role 이 필요 이상으로 넓을 때
- **예:** 특정 컬렉션에 `find` 와 `insert` 만 주고 `remove` 는 빼기

최소 권한 원칙이 실제로 적용되는 자리입니다.

「이 계정이 잘못 쓰이면 영향 범위가 어디까지인가」로 범위를 정합니다.

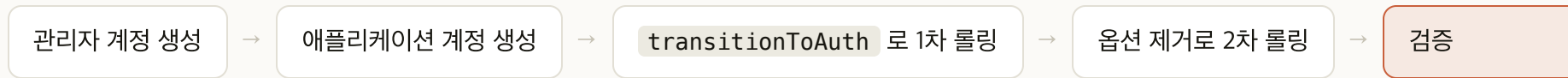
keyFile 내부 인증

항목	내용
대상	사용자가 아니라 노드 간 인증 . 세 멤버가 서로를 확인하는 공유 비밀
배치 상태	세 노드와 ops 노드의 <code>/etc/mongod.key</code> 에 이미 같은 파일이 있음
켜지는 시점	<code>mongod.conf</code> 에 <code>security.keyFile</code> 을 넣는 순간. 인가도 함께 켜짐
권한	<code>600</code> . 아니면 <code>mongod</code> 가 기동을 거부
세 노드의 동일성	파일이 다르면 멤버가 서로를 거부

계정과 암호로는 노드 간 통신이 서지 않습니다.

`security.keyFile` 한 줄이 클라이언트 인증과 노드 간 인증을 함께 켭니다. 계정을 먼저 만들어 두어야 하는 이유가 여기 있습니다.

인증 활성화 순서



- **롤링이 두 번인 이유:** 인증이 꺼진 노드와 켜진 노드가 섞이면 하트비트가 깨짐. `transitionToAuth` 는 인증 없는 연결도 함께 받아 그 구간을 건너게 함
- **동시 재시작 금지:** 세 멤버를 한꺼번에 내리면 과반을 잃어 Replica Set 이 멈춤

순서를 지키지 않으면 아무도 접속할 수 없습니다.

관리자 계정을 만들기 전에 인증을 켜면 작업자 본인도 접속하지 못합니다.

계정 생성

```
// 인증을 켜기 전에 PRIMARY 의 admin 에서 실행합니다
mongosh "mongodb://<PRIMARY>:27017/admin"
db.createUser({ user: "admin", pwd: passwordPrompt(), roles:
  ["userAdminAnyDatabase", "clusterAdmin", "readWriteAnyDatabase"] })
db.createUser({ user: "appuser", pwd: passwordPrompt(), roles:
  [{ role: "readWrite", db: "lab" }] })
db.createUser({ user: "exporter", pwd: passwordPrompt(), roles: ["clusterMonitor"] })
```

- **확인 지점:** `db.getUsers().map(u => u.user)` 에 세 계정이 보임
- **범위:** `admin` 은 전체, `appuser` 는 `lab`, `exporter` 는 지표 조회

인증을 켜기 전에 만들어야 합니다. 계정이 없으면 접속할 방법이 없어집니다.

사용자 정의 Role 정의

```
// admin 에서. 컬렉션 하나에 대한 조회만 허용하는 Role
db.createRole({ role: "orderReader", roles: [], privileges:
  [{ resource: { db: "lab", collection: "orders" }, actions: ["find"] }] })
// 정의 확인
db.getRole("orderReader", { showPrivileges: true })
```

- **범위 단위:** 데이터베이스가 아니라 **컬렉션 하나**. `resource` 에 `collection` 을 적으면 그 컬렉션에만 걸림
- **액션 단위:** `find` 만 허용. `insert` · `update` · `remove` 는 빠짐
- **roles: []** : 다른 Role 을 상속하지 않음. 여기 적은 권한이 전부

인증 활성화 롤링

```
# ops 노드의 ~/ansible 에서. 1차. 인증 있는 연결과 없는 연결을 모두 받습니다
cd ~/ansible
ansible-playbook play-mongodb.yml -e mongodb_replset_enabled=true \
    -e mongodb_auth_enabled=true -e mongodb_transition_to_auth=true
# 2차. transition 만 빼고 다시 돌려 인증을 강제합니다
ansible-playbook play-mongodb.yml -e mongodb_replset_enabled=true \
    -e mongodb_auth_enabled=true
```

이후의 모든 명령에 자격 증명이 붙습니다.

`mongosh "mongodb://<노드>:27017/lab" -u <계정> -p --authenticationDatabase admin` 형태입니다. 붙이지 않으면 챕터 11과 챕터 12의 명령이 전부 거부됩니다.

exporter 재구성

```
# 인증을 켜 뒤 exporter 에 자격 증명을 넣어 다시 배포합니다
ansible-playbook play-mongodb.yml -e mongodb_replset_enabled=true \
  -e mongodb_auth_enabled=true \
  -e mongodb_exporter_uri=mongodb://exporter:<암호>@127.0.0.1:27017/?authSource=admin
```

- **확인 지점:** Prometheus 의 Targets 에서 `mongodb` 대상 3개가 `UP`
- **authSource=admin :** `exporter` 계정이 `admin` 에 있으므로 인증 대상 데이터베이스를 지정
- **127.0.0.1 로 붙는 이유:** exporter 가 각 노드 안에서 자기 `mongod` 를 봄

이 단계를 빠뜨리면 관측이 통째로 끊깁니다.

인증을 켜 순간 자격 증명이 없는 exporter 는 지표를 읽지 못하고 `mongodb` 대상 3개가 전부 `DOWN` 이 됩니다. 인증을 켜는 작업과 관측을 되살리는 작업은 한 묶음입니다.

권한 밖 작업의 반응

```
// 1. 자격 증명 없이 접속해 조회
mongosh "mongodb://<PRIMARY>:27017/lab" --eval 'db.orders.countDocuments()'
// 2. appuser 로 admin 데이터베이스 접근
mongosh "mongodb://<PRIMARY>:27017/admin" -u appuser -p --authenticationDatabase admin
db.system.users.find()
```

```
MongoServerError: Command aggregate requires authentication
MongoServerError: not authorized on admin to execute command
{ find: "system.users", ... }
```

거부되는 것을 직접 확인해야 권한이 적용됐다고 말할 수 있습니다. 같은 `appuser` 로 `lab.orders` 를 세면 200000 이 나옵니다.

11

PRIMARY 정지와 선출

정지

선출

복귀

Rollback

정지 전 준비

```
// 현재 PRIMARY 확인
rs.status().members.map(m => [m.name, m.stateStr])
// Rollback 확인 대상 문서 하나를 w:1 로 씁니다
db.probe.insertOne({ tag: "before-stop" }, { writeConcern: { w: 1 } })
```

- **노드 이름 찾기:** `rs.status()` 는 IP 를 돌려주므로 마지막 자리로 매핑. 10.0.1.5 가 `mongo-1` , 10.0.1.6 이 `mongo-2` , 10.0.1.7 이 `mongo-3`
- **w:1 로 쓰는 이유:** 과반이 받았는지 확인하지 않고 응답. 되돌려질 수 있는 상태를 만드는 것이 목적
- **적어 둘 것:** 현재 PRIMARY 의 노드 이름과 IP

PRIMARY 정지

```
# ops 노드의 ~/ansible 에서
cd ~/ansible
ansible <PRIMARY 노드 이름> -m systemd -a "name=mongod state=stopped" --become
```

- 명령이 바로 돌아오지 않음: `mongod` 가 완전히 내려갈 때까지 약 15초 걸림
- 전환은 그 사이에 끝남: `systemctl stop` 은 정상 종료라 `mongod` 가 내려가기 시작하면서 곧바로 자리를 넘김. 1초 안에 끝남
- 다른 창에서 관찰: 명령을 기다리는 동안 남은 노드에 접속해 상태를 봄

무응답으로 판정되기를 기다리는 경로가 아닙니다.

정지 노드가 스스로 물러나므로 `electionTimeoutMillis` 만큼 기다리지 않습니다. 전원이 끊기거나 네트워크가 끊기는 장애는 이보다 오래 걸립니다.

선출 확인

```
// 남아 있는 노드에 접속해 반복 실행합니다
rs.status().members.map(m => [m.name, m.stateStr, m.health])
// 선출이 끝난 뒤 쓰기
db.probe.insertOne({ tag: "after-election" })
```

보는 것

판정

정지시킨 멤버

health: 0 과 (not reachable/healthy)

남은 두 멤버

과반 2표로 하나가 PRIMARY 로 전환

선출 중 쓰기

not primary 계열 오류. 구간이 짧아 놓치기 쉬움

선출 후 쓰기

새 PRIMARY 가 수용

정지 노드 복귀

```
# 정지시킨 노드를 다시 올립니다
ansible <정지시킨 노드 이름> -m systemd -a "name=mongod state=started" --become
```

```
// 합류와 복제 지연 확인
rs.status().members.map(m => [m.name, m.stateStr])
rs.printSecondaryReplicationInfo()
```

원래 PRIMARY 는 자동으로 자리를 되찾지 않습니다.

SECONDARY 로 합류합니다. 특정 노드를 PRIMARY 로 되돌리려면 `priority` 를 조정하거나 현재 PRIMARY 에서 `rs.stepDown()` 을 실행합니다.

Rollback 과 정지 시간 판정

```
# 되살린 노드 안에서. rollback 디렉터리가 생겼는지 봅니다
ansible <노드 이름> -m shell -a "ls -l /var/lib/mongodb/rollback 2>&1" --become
```

```
// 정지 시간이 재생으로 따라잡을 수 있는 범위였는지
db.getReplicationInfo().timeDiff
```

확인 대상	판정
rollback 디렉터리	없어야 정상. 정상 종료라 되돌릴 것이 남지 않음
before-stop 문서	그대로 남아 있음
정지 시간과 oplog 윈도우	윈도우 안이면 재생으로 복귀, 넘으면 초기 동기화

12

Sharding으로 확장

Sharding 구조

구성 요소

분할 방식

Shard Key

전환 절차

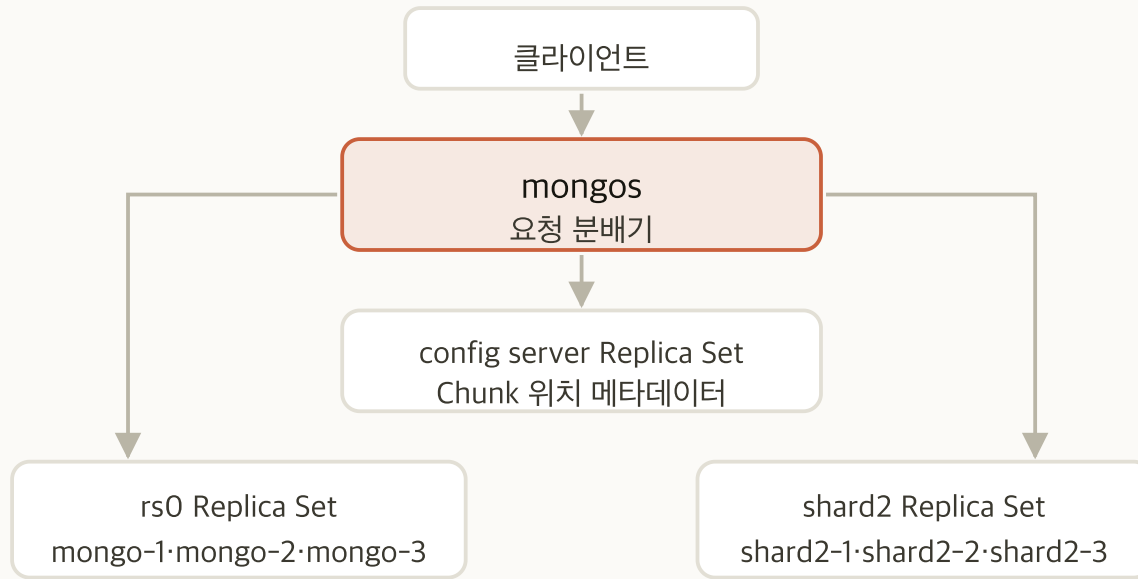
Sharding의 정의

- **구성:** 컬렉션의 데이터를 Shard Key 기준으로 여러 Replica Set 에 나눠 담음
- **확장 효과:** 담을 수 있는 양과 쓰기 처리량이 Shard 수만큼 늘어남
- **복제와의 관계:** 각 Shard 가 그 자체로 Replica Set

복제 위에 분할이 더해지는 구조입니다.

Shard 하나가 통째로 정지하면 그 조각을 잃습니다. 가용성은 여전히 각 Shard 의 복제가 맞습니다.

Sharded Cluster의 구성 요소



- **mongos** : 요청을 어느 Shard 로 보낼지 정함. 상태를 갖지 않음
- **config server**: 어느 Chunk 가 어느 Shard 에 있는지를 담음
- **clusterRole** : Shard 는 **shardsvr** , config server 는 **configsvr** 로 기동. 기존 Replica Set 을 편입할 때도 다시 세움
- **추가 여섯 대**: **Standard_D2ds_v5** . 기본 캐시 3.5 GiB 로 충분한 규모

접속 대상이 바뀝니다.

Replica Set 에 직접 접속하던 것을 **mongos** 를 거치도록 바뀌야 합니다. 애플리케이션의 연결 문자열이 함께 바뀝니다.

분할 방식

방식	나누는 기준	성질
Ranged	Shard Key 값의 범위	범위 조회가 한 Shard 로 몰림. 값이 단조 증가하면 마지막 Shard 에 쓰기가 쏠림
Hashed	Shard Key 의 해시값	쓰기가 고르게 흩어짐. 범위 조회가 전 Shard 로 뿌려짐

- **Zone:** 두 방식 위에 얹어 특정 Shard Key 범위를 특정 Shard 에 고정하는 배치 정책. 단독으로 고르는 방식이 아님
- **이동 중의 조회:** `readConcern: available` 로 읽으면 아직 정리되지 않은 문서까지 보임

쓰기 분산과 범위 조회는 맞바꾸는 관계입니다.

Hashed 는 쓰기를 흩뜨리는 대신 범위 조회를 전 Shard 로 뿌립니다.

Shard Key 선택 기준

항목	내용
되돌리기	한 번 정한 뒤 바꾸려면 큰 작업이 필요
값의 가짓수	값의 가짓수가 적으면 Chunk 를 더 쪼갤 수 없음
분포	값이 치우치면 한 Shard 에 쓰기가 몰림
질의 패턴	Shard Key 를 포함하지 않는 조회는 모든 Shard 로 뿌려짐

자주 쓰는 조회 조건이 Shard Key 에 들어 있어야 합니다.
그렇지 않으면 Shard 를 늘려도 조회 부하가 모든 Shard 로 그대로 퍼집니다.

Shard Key 선택 실패의 증상

잘못된 선택	나타나는 증상
단조 증가 값 (예: 시각)	마지막 Chunk 에 쓰기가 쏠려 Shard 한 대에만 부하가 걸림
가짓수가 적은 값 (예: 상태 코드)	Chunk 를 더 쪼갤 수 없어 데이터가 고르게 퍼지지 않음
조회에 쓰지 않는 필드	모든 조회가 전 Shard 로 뿌려짐

한 Shard 에만 부하가 몰리는 상태를 Hot Shard 라고 부릅니다.
노드를 늘려도 처리량이 오르지 않아 Sharding 의 효과가 없습니다.

Sharding 노드 추가

```
# 자기 PC 의 lab/terraform/02-mongodb 에서. ops 노드가 아닙니다
# session.tfvars 에 한 줄을 더합니다
sharding_enabled = true
terraform apply -var-file=session.tfvars
```

- 추가되는 여섯 대: `cfg-1` · `cfg-2` · `cfg-3` (10.0.1.8·9·10), `shard2-1` · `shard2-2` · `shard2-3` (10.0.1.11·12·13)
- 기존 노드는 그대로: `rs0` 세 대와 ops 노드는 다시 만들어지지 않음
- 확인 지점: `apply` 출력의 `Plan` 에 `destroy` 가 0 인지

인벤토리에 두 그룹 추가

```
# ops 노드의 ~/ansible/inventory/hosts.ini 끝에 덧붙입니다
[configsvr]
cfg-1    ansible_host=10.0.1.8
cfg-2    ansible_host=10.0.1.9
cfg-3    ansible_host=10.0.1.10
[shard2]
shard2-1 ansible_host=10.0.1.11
shard2-2 ansible_host=10.0.1.12
shard2-3 ansible_host=10.0.1.13
```

- **확인 지점:** `cd ~/ansible && ansible configsvr,shard2 -m ping` 이 여섯 대 모두 `pong`
- **[mongo]** 그룹은 건드리지 않음: `rs0` 세 대가 그대로 있어야 함

새 노드 구성

```
cd ~/ansible
ansible-playbook play-mongodb-shard.yml
```

- 이 플레이북이 하는 일 들: 새 여섯 대에 `mongod` 를 올리고, 기존 `rs0` 세 대를 `shardsvr` 로 전환
- `rs0` 도 재시작됨: `clusterRole: shardsvr` 이 들어가면서 롤링으로 다시 뜸
- 확인 지점: `rs.status()` 가 여전히 PRIMARY 1 SECONDARY 2

포트는 27017 로 고정돼 있습니다.

`clusterRole` 을 주면 기본 포트가 27018 과 27019 로 바뀌지만 `net.port` 로 고정했습니다. 명령의 포트가 앞 챕터와 같습니다.

config server 초기화

```
# ops 에서 cfg-1 로 들어가 그 노드 안에서 실행합니다
ssh 10.0.1.8
mongosh
```

```
// configsvr: true 가 없으면 초기화를 거부합니다
rs.initiate({ _id: "cfg", configsvr: true, members: [
  { _id: 0, host: "10.0.1.8:27017" },
  { _id: 1, host: "10.0.1.9:27017" },
  { _id: 2, host: "10.0.1.10:27017" } ] })
```

- **노드 안에서 실행하는 이유:** 인증을 켜 상태라 외부에서 계정 없이 초기화할 수 없음. 초기화 전에는 localhost 예외가 열려 있음
- **확인 지점:** `rs.status().set` 이 `cfg` 이고 PRIMARY 가 하나

shard2 초기화

```
# ops 로 나온 뒤 shard2-1 로 들어갑니다
ssh 10.0.1.11
mongosh
```

```
// 두 번째 Shard 는 일반 Replica Set 입니다. configsvr 을 주지 않습니다
rs.initiate({ _id: "shard2", members: [
  { _id: 0, host: "10.0.1.11:27017" },
  { _id: 1, host: "10.0.1.12:27017" },
  { _id: 2, host: "10.0.1.13:27017" } ] })
```

Shard 를 늘리는 것은 Replica Set 을 하나 더 만드는 것입니다.

mongos 기동과 관리자 계정

```
# ops 노드에서. keyFile 이 없으면 Shard 에 접속하지 못합니다
mongos --configdb cfg/10.0.1.8:27017 --keyFile /etc/mongod.key \
  --port 27020 --fork --logpath ~/mongos.log
```

```
// 자격 증명 없이 접속해 cfg 에 클러스터 관리자를 만듭니다
mongosh --port 27020
db.getSiblingDB("admin").createUser({ user: "csadmin",
  pwd: passwordPrompt(), roles: ["root"] })
```

챕터 10의 **admin** 계정으로는 **mongos** 에 접속하지 못합니다.

그 계정은 **rs0** 안에만 있습니다. 클러스터의 계정은 config server 가 들고 있으므로 여기서 새로 만듭니다.

Shard 등록

```
// csadmin 으로 mongos 에 접속한 뒤
sh.addShard("rs0/10.0.1.5:27017")
sh.addShard("shard2/10.0.1.11:27017")
sh.status()
```

- 주소 형식: <세트 이름>/<멤버 하나의 주소>. 나머지 멤버는 세트 정보에서 자동으로 채워짐
- 확인 지점: `sh.status()` 의 `shards` 에 `rs0` 와 `shard2` 두 항목
- 기존 데이터: `rs0` 의 `lab` 이 그대로 남아 있고 `primary: 'rs0'` 로 잡힘

Chunk 크기와 Shard Key 인덱스

```
// Chunk 크기를 1 MB 로 낮춥니다
db.getSiblingDB("config").settings.updateOne(
  { _id: "chunksize" }, { : { value: 1 } }, { upsert: true })
// Shard Key 필드에 인덱스가 있어야 합니다
use lab
db.orders.createIndex({ customerId: 1 })
```

- 크기를 낮추는 이유: 기본값 128 MB 가 이 적재량보다 커서 컬렉션이 한 덩어리로 남음. 나뉘는 것을 보려면 낮춰야 함
- **customerId** 를 고른 이유: 값이 5,000 가지라 카디널리티가 있고 단조 증가가 아님

운영에서 Chunk 크기를 낮추면 이동이 잦아집니다.

Chunk 를 옮기는 동안 원본과 대상 양쪽에 부하가 걸립니다. 이 값은 실습에서 분할을 보기 위한 설정입니다.

Sharding 활성화와 분포 확인

```
sh.enableSharding("lab")
sh.shardCollection("lab.orders", { customerId: 1 })
sh.status()
```

```
shards:
  { _id: 'rs0',    host: 'rs0/10.0.1.5:27017,...' }
  { _id: 'shard2', host: 'shard2/10.0.1.11:27017,...' }
databases:
  { _id: 'lab', primary: 'rs0' }
    lab.orders
      shard key: { customerId: 1 }
      chunks:  rs0 2,  shard2 17
```

분포가 고르지 않은 것이 정상입니다. balancer 가 재배치하는 중이라 19개가 2개와 17개로 갈렸습니다. 두 Shard 에 나뉘었다는 것만 확인합니다. Chunk 가 하나로 남아 있으면 chunksize 를 낮추는 단계를 건너뛴 것입니다.

Shard Key 유무에 따른 조회

```
// 1. Shard Key 를 포함한 조회
db.orders.find({ customerId: "C000123" }).explain().queryPlanner.winningPlan.stage
// 2. Shard Key 가 없는 조회
db.orders.find({ status: "refunded", amount: { : 90000 } })
    .explain().queryPlanner.winningPlan.stage
```

조회	단계	대상
Shard Key 포함	SINGLE_SHARD	한 Shard 만
Shard Key 없음	SHARD_MERGE	모든 Shard 에 보내고 합침

Shard Key 설계가 곧 조회 비용입니다. 가장 잦은 조회가 Shard Key 를 포함하도록 고릅니다.

리소스 정리

```
# 남길 것을 먼저 각자 PC 로 옮깁니다
scp -i ~/.ssh/rkm -r azureuser@<ops 공인 IP>:~/dump .
```

```
# 자기 PC 의 lab/terraform/02-mongodb 에서 실행. ops 노드 아님
terraform destroy -var-file=session.tfvars
```

destroy 는 리소스 그룹과 그 안의 자원을 전부 삭제합니다.
되돌릴 수 없습니다. 반출할 것을 먼저 옮긴 뒤에 실행하세요.